PROJETO CLIMAALERTA — MATERIAL PEDAGÓGICO

Manual Pedagógico do ClimaAlerta

Desenvolvimento passo a passo no PyCharm: do ambiente Python à integração, testes, depuração, Git e apresentação

Documento	MAN-PED
Versão	1.1 (PyCharm)
Data de geração	21/10/2026 16:46
Software	ClimaAlerta 1.0 — monitoramento meteorológico educacional
Público	Alunos do Ensino Médio e do Curso Técnico em Informática; professores
Situação	Tutorial executável: o código foi testado automaticamente (tests/test_tutorial.py)

Projeto educacional — Ensino Médio e Curso Técnico em Informática.

Fontes de dados: Open-Meteo (CC BY 4.0), INMET, IBGE. Gerado por scripts/gerar_documentacao.py.

Sumário

Apresentação 3
Módulo 1 — Preparação do ambiente 7
Módulo 2 — Fundamentos de Python 12
Módulo 3 — Primeira dashboard com Streamlit 16
Módulo 4 — Fundamentos de HTML e seletores CSS 21
Módulo 5 — Web Scraping com Selenium 24
Módulo 6 — Consumo de APIs 29
Módulo 7 — Tratamento de dados com Pandas 33
Módulo 8 — Modelagem e persistência com SQLite e SQLAlchemy 37
Módulo 9 — Implementação das regras de negócio 41
Módulo 10 — Desenvolvimento da interface 45
Módulo 11 — Integração dos módulos 49
Módulo 12 — Atualização automatizada 55
Módulo 13 — Testes e depuração no PyCharm 59
Módulo 14 — Documentação e versionamento com Git no PyCharm 63
Módulo 15 — Integração final e apresentação do projeto 67
Revisão geral 72
Glossário 72
Referências 73

Apresentação

Este manual conduz você, passo a passo, pelo desenvolvimento do **ClimaAlerta**, um sistema de monitoramento meteorológico em Python, usando a IDE **PyCharm**. Você começará preparando o computador e escrevendo pequenos programas; aprenderá a criar dashboards, extrair dados de páginas web com Selenium, consumir APIs, tratar dados com Pandas, gravar em banco de dados, implementar regras de negócio, testar, depurar, documentar e, ao final, integrar e apresentar o sistema completo.

Todos os arquivos que você vai criar estão **completamente comentados**: cada arquivo começa explicando o que faz, como executá-lo no PyCharm e quais conceitos usa, e cada trecho importante tem comentários explicando o porquê.

Como o manual está organizado

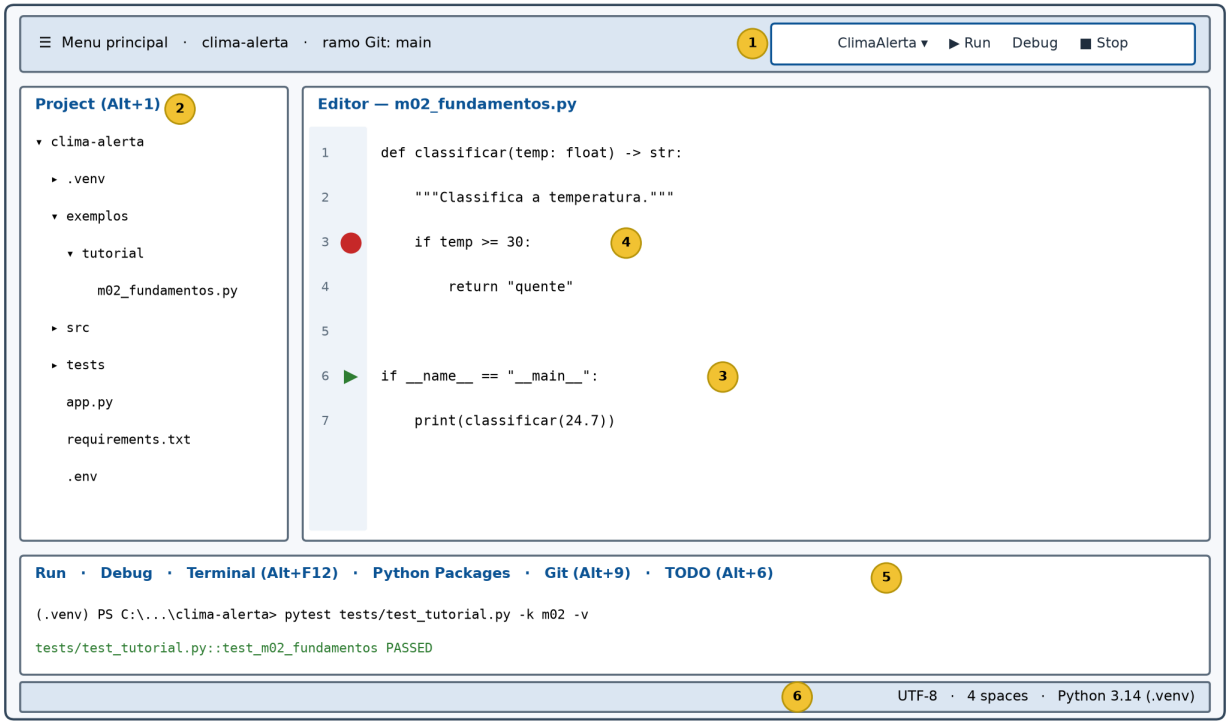
São **15 módulos**. Cada módulo tem sempre as mesmas 13 seções:

Nº	Seção	Para que serve
1	Objetivo de aprendizagem	O que você saberá fazer ao final
2	Pré-requisitos	O que precisa estar pronto antes
3	Conceitos	Explicação da teoria necessária
4	Arquivos a criar ou alterar	Caminho exato de cada arquivo
5	Passo a passo no PyCharm	Ações na IDE e comandos equivalentes no Terminal
6	Código	Código completo e comentado (o mesmo testado pelo professor)
7	Explicação do código	Resumo do que cada parte faz
8	Resultado esperado	O que deve aparecer na tela
9	Verificação	Como comprovar que deu certo (geralmente um teste automático)
10	Erros frequentes	Problemas comuns e soluções
11	Exercícios práticos	Para fixar o conteúdo
12	Critérios de conclusão	Checklist para avançar
13	Desafios de extensão	Para quem terminou antes

Conhecendo o PyCharm

O PyCharm é um **ambiente de desenvolvimento integrado (IDE)**: reúne editor de código, execução, testes, depurador, terminal e Git em uma única janela. Este manual foi preparado com o **PyCharm 2025.3**. Os recursos usados no curso fazem parte da versão gratuita; a JetBrains também oferece licença educacional gratuita para alunos e professores, que libera recursos extras (como a janela de banco de dados). Em versões diferentes, os nomes de alguns menus podem variar um pouco.

Janela do PyCharm — regiões usadas no curso (ilustração esquemática)



1 Configurações de execução · 2 Janela Project · 3 ▶ executar o arquivo · 4 ponto de parada · 5 janelas inferiores (Run, Terminal...) · 6 interpretador do projeto

Figura 1 — Regiões da janela do PyCharm (ilustração esquemática, não é captura de tela)

Região	Para que serve	Como abrir
Janela **Project**	Ver e criar pastas e arquivos do projeto	Alt+1
Editor	Escrever o código; a margem esquerda mostra ▶ (executar) e pontos de parada	Duplo clique em um arquivo
Janela **Run**	Saída dos programas executados	Abre sozinha ao executar
Terminal	Linha de comando já dentro da pasta do projeto e com o `.venv` ativo	Alt+F12
Python Packages	Pesquisar, instalar e atualizar bibliotecas	View > Tool Windows > Python Packages
Debug	Depurar: pausar o programa e inspecionar variáveis	Botão ou Shift+F9
Git / Commit	Histórico e envio de alterações	Alt+9 / Alt+0
TODO	Lista de todos os comentários `TODO` do projeto	Alt+6
Barra de status	Mostra (e troca) o interpretador Python do projeto	Canto inferior direito

Atalhos mais usados (teclado padrão do Windows/Linux):

Ação	Atalho
Executar o arquivo aberto	Ctrl+Shift+F10
Executar de novo a última configuração	Shift+F10
Depurar a última configuração	Shift+F9
Parar a execução	Ctrl+F2
Procurar qualquer coisa (arquivos, ações, configurações)	Shift duas vezes
Configurações (Settings)	Ctrl+Alt+S
Documentação do nome sob o cursor	Ctrl+Q
Ir para a definição	Ctrl+B ou Ctrl+clique
Reformatar o código	Ctrl+Alt+L

Ação	Atalho
Comentar/descomentar linhas	Ctrl+/
Sugestões de correção (lâmpada)	Alt+Enter

Convenções deste manual

- **Pasta raiz do projeto:** a pasta que contém `app.py`, `src/`, `tests/` e `exemplos/`. Ela aparece no topo da janela Project.
- **Executar com ▶:** clicar no triângulo verde que aparece na margem esquerda do editor, ao lado da linha `if __name__ == "__main__":`, ou clicar com o botão direito no editor e escolher **Run '<nome do arquivo>'**.
- **Terminal do PyCharm:** os blocos de comandos indicam "Terminal do PyCharm". Ele abre na pasta raiz do projeto e já ativa o ambiente virtual. No Windows o terminal é o PowerShell; no Linux, o Bash. Quando um comando for diferente nos dois sistemas, o manual mostra as duas versões.
- **Pasta de trabalho:** configuraremos o PyCharm (módulo 1) para que todos os programas e testes rodem a partir da pasta raiz. Assim, caminhos como `exemplos/tutorial/...` funcionam igual no ▶ e no Terminal.
- Os dados meteorológicos reais vêm da **Open-Meteo** (licença CC BY 4.0) e do **INMET**. Dados **fictícios** ou **simulados** são sempre identificados.
- As saídas mostradas em "Resultado esperado" foram obtidas executando o código em 21/10/2026. Temperaturas atuais mudarão quando você executar.

> **Importante:** O ClimaAlerta é um projeto de estudo. Os "indicadores locais" que você vai programar **não são alertas oficiais**. Em situações de risco, siga o INMET e a Defesa Civil (199).

Sequência e carga horária sugerida

Módulo	Tema	Aulas sugeridas (50 min)
1	Preparação do ambiente (PyCharm, Python, dependências)	3
2	Fundamentos de Python	3
3	Primeira dashboard com Streamlit	2
4	HTML e seletores CSS	2
5	Web Scraping com Selenium	3
6	Consumo de APIs	2
7	Tratamento de dados com Pandas	3
8	SQLite e SQLAlchemy	3
9	Regras de negócio	2
10	Desenvolvimento da interface	3
11	Integração dos módulos	3
12	Atualização automatizada	2
13	Testes e depuração no PyCharm	3
14	Documentação e Git no PyCharm	3
15	Integração final e apresentação	4

Mapa do aprendizado

Nos módulos 1 a 10 você escreve **seus próprios arquivos** na pasta `exemplos/tutorial/`, um por módulo. Cada um é uma versão pequena e didática de uma parte do sistema. Nos módulos 11 a 15 você estuda, integra e estende o **ClimaAlerta completo**, comparando cada parte com o que escreveu.

Seu arquivo (módulos 1–13)	Parte correspondente no ClimaAlerta completo
`m02_fundamentos.py`	`src/utils/` (funções de conversão e validação)
`m03_primeira_dashboard.py`	`exemplos/dashboard_python/app.py` e `src/ui/`
`m04_pagina.html`	`exemplos/selenium/pagina_meteorologica_demo.html`
`m05_selenium_basico.py`	`src/scraper/`
`m06_api_open_meteo.py`	`src/integrations/open_meteo.py`
`m07_pandas.py`	`src/services/weather_service.py`, `export_service.py`
`m08_banco.py`	`src/database/`
`m09_regras.py`	`src/services/indicators.py`
`m10_interface.py`	`src/ui/page_*.py`
`m12_agendador.py`	`src/collectors/scheduler.py`
`test_m09_regras.py`	`tests/`

Módulo 1 — Preparação do ambiente

1.1 Objetivo de aprendizagem

Instalar o Python, o Google Chrome e o PyCharm; abrir o projeto no PyCharm; criar o **ambiente virtual** e configurá-lo como interpretador do projeto; **instalar as dependências** do `requirements.txt`; ajustar as configurações do PyCharm usadas no curso; e confirmar que tudo funciona executando um programa de verificação.

1.2 Pré-requisitos

- Computador com Windows 10/11 ou Linux e acesso à internet.
- Permissão para instalar programas (ou ajuda do técnico do laboratório).
- A pasta do projeto ClimaAlerta, entregue pelo professor (cópia local ou repositório do GitHub).

1.3 Conceitos

- **Interpretador Python:** o programa que lê e executa os arquivos `.py`. Usaremos a versão 3.12 ou mais nova.
- **IDE (PyCharm):** o ambiente onde escrevemos, executamos, testamos e depuramos o código.
- **Ambiente virtual (`.venv`):** uma cópia isolada do Python dentro do projeto, com as bibliotecas daquele projeto. Evita conflitos entre projetos e não exige permissão de administrador. No PyCharm, ele é o **interpretador do projeto**.
- **Dependências:** bibliotecas de terceiros que o projeto usa (Streamlit, Pandas, Selenium...). O arquivo `requirements.txt` lista cada uma com a versão exata; o **pip** é o instalador.
- **Run Configuration (configuração de execução):** "receita" que o PyCharm guarda para executar um programa: qual arquivo ou módulo, com quais parâmetros e em qual pasta de trabalho.

1.4 Arquivos a criar ou alterar

Arquivo	Ação
`.venv/`	Criado pelo PyCharm (não edite)
`.env`	Cópia de `.env.example` com as configurações
`exemplos/tutorial_referencia/`	Novo nome da pasta de soluções do professor (passo 7)
`exemplos/tutorial/`	Pasta **nova e vazia** onde você escreverá seus arquivos
`exemplos/tutorial/m01_verificar_ambiente.py`	Seu primeiro programa

1.5 Passo a passo no PyCharm

Passo 1 — Instalar o Python. No Windows, baixe em python.org e, na primeira tela do instalador, marque **"Add python.exe to PATH"**. No Linux (Ubuntu/Debian), no terminal do sistema:

Bash — terminal do sistema (Linux)

```
sudo apt install python3 python3-venv python3-pip
```

Passo 2 — Instalar o Google Chrome (usado no módulo 5) e o **PyCharm** (site jetbrains.com/pycharm). No Windows, durante a instalação do PyCharm, marque "Add 'bin' folder to the PATH" e "Create Desktop Shortcut".

Passo 3 — Abrir o projeto. Na tela inicial do PyCharm, clique em **Open** e escolha a **pasta raiz** do projeto (a que contém `app.py`). Se o PyCharm perguntar se você confia no projeto, escolha

Trust Project. Se o projeto estiver no GitHub, use **Clone Repository** e cole o endereço do repositório.

Passo 4 — Criar o ambiente virtual e defini-lo como interpretador. O PyCharm pode oferecer isso automaticamente: se aparecer uma faixa amarela ou um aviso no canto inferior direito dizendo que não há interpretador configurado, clique em **Configure Python interpreter**. Caso contrário, faça manualmente:

1. Abra **Settings** (Ctrl+Alt+S) > **Project: <nome do projeto>** > **Python Interpreter**.
2. Clique em **Add Interpreter** > **Add Local Interpreter...**
3. Escolha **Generate new** (gerar novo), tipo **Virtualenv**.
4. **Base python:** o Python 3.12 ou mais novo instalado no passo 1.
5. **Location:** `<pasta raiz>\.venv` (Windows) ou `<pasta raiz>/.venv` (Linux).
6. Clique em **OK**. A barra de status (canto inferior direito) passa a mostrar algo como `Python 3.14 (.venv)`.

> **Nota:** Se a pasta `.venv` já existir (criada antes), use **Select existing** e aponte para `.venv\Scripts\python.exe` (Windows) ou `.venv/bin/python` (Linux).

Passo 5 — Instalar as dependências. Escolha **um** dos três caminhos:

- **Caminho A — aviso do `requirements.txt` (recomendado):** abra o arquivo `requirements.txt` com duplo clique na janela Project. O PyCharm compara a lista com o que está instalado e mostra uma faixa no topo do editor com a mensagem de que os requisitos não estão satisfeitos e o link **Install requirements**. Clique nele, mantenha todos os pacotes marcados e confirme em **Install**. O progresso aparece na barra de status; aguarde terminar (alguns minutos).
- **Caminho B — janela Python Packages:** abra **View > Tool Windows > Python Packages**. Pesquise cada pacote pelo nome (por exemplo, `streamlit`), selecione a versão indicada no `requirements.txt` e clique em **Install**. Útil para instalar um pacote isolado depois.
- **Caminho C — Terminal do PyCharm:** abra o Terminal (Alt+F12) e execute:

Terminal do PyCharm — pasta raiz (Windows e Linux)

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Para conferir, volte a **Settings > Project > Python Interpreter**: a lista deve mostrar `streamlit`, `pandas`, `selenium`, `SQLAlchemy`, `pytest` e as demais bibliotecas com as versões do `requirements.txt`.

> **Dica:** Sempre que alguém acrescentar uma biblioteca ao `requirements.txt`, abra o arquivo de novo: o PyCharm volta a mostrar **Install requirements** apenas para o que falta.

Passo 6 — Ajustes do PyCharm usados no curso (uma vez por projeto).

1. **Codificação UTF-8:** Settings > Editor > **File Encodings** — Global, Project e Default encoding for properties files em **UTF-8**. Evita problemas com acentos.
2. **pytest como executor de testes:** Settings > Tools > **Python Integrated Tools** > Testing > Default test runner: **pytest**.
3. **Pasta de trabalho padrão:** menu **Run > Edit Configurations...** > link **Edit configuration templates...** > **Python** > Working directory: `$ProjectFileDir$`. Repita em **Python tests > pytest**. Assim todos os programas e testes executam a partir da pasta raiz.
4. **Arquivo `.env`:** clique com o botão direito em `.env.example` > **Copy** (Ctrl+C), clique na pasta raiz > **Paste** (Ctrl+V) e dê o nome `.env`.

Passo 7 — Preparar a pasta do tutorial. O projeto já traz, em `exemplos/tutorial/`, as soluções de referência. Na janela Project, clique com o botão direito em `exemplos/tutorial` >

Refactor > Rename (Shift+F6) e digite `tutorial_referencia`. Depois, clique com o botão direito em `exemplos` > **New > Directory** e crie `tutorial`. Por fim, clique com o botão direito na nova pasta `tutorial` > **Mark Directory as > Sources Root** (faz o editor reconhecer as importações entre os seus arquivos).

> **Nota:** No repositório-esqueleto distribuído pelo professor, a pasta `exemplos/tutorial/` já vem vazia: pule a renomeação e faça apenas o "Mark Directory as".

Passo 8 — Criar e executar o primeiro arquivo. Clique com o botão direito em `exemplos/tutorial` > **New > Python File**, digite `m01_verificar_ambiente` (sem `.py`) e Enter. Digite o código da seção 1.6 e execute com o ▶ verde ao lado de `if __name__ == "__main__":`. A saída aparece na janela **Run**.

1.6 Código

exemplos/tutorial/m01_verificar_ambiente.py

```
"""Módulo 1 — Verifica se o ambiente de desenvolvimento está pronto para o projeto.
O QUE ESTE ARQUIVO FAZ
    Mostra a versão do Python em uso, confere se o ambiente virtual (.venv) está
    ativo e lista a versão de cada biblioteca do projeto. No final, informa se
    está tudo pronto ou o que precisa ser corrigido.
COMO EXECUTAR NO PYCHARM
    1. Abra este arquivo no editor (duplo clique na janela Project, à esquerda).
    2. Clique no triângulo verde ▶ ao lado da linha  if __name__ == "__main__":
       ou clique com o botão direito no editor e escolha  Run 'm01_verificar_ambiente'.
       Atalho: Ctrl+Shift+F10.
    3. O resultado aparece na janela Run, na parte de baixo da tela.
    Alternativa pelo Terminal do PyCharm (Alt+F12), na pasta raiz do projeto:
        python exemplos/tutorial/m01_verificar_ambiente.py
CONCEITOS USADOS
    import, listas, funções, try/except, f-strings e o bloco  if __name__ == "__main__":
"""
# ---------------------------------------------------------------------------
# IMPORTAÇÕES
# "import" traz para o nosso programa ferramentas que já existem no Python.
# ---------------------------------------------------------------------------
import sys  # "sys" dá acesso a informações sobre o interpretador Python em execução
from importlib import metadata  # "metadata" consulta os pacotes instalados e suas versões
# ---------------------------------------------------------------------------
# CONSTANTE
# Uma lista com o nome dos pacotes que o projeto precisa. Por convenção, nomes
# escritos TODOS EM MAIÚSCULAS indicam valores que não devem mudar (constantes).
# ---------------------------------------------------------------------------
PACOTES = ["requests", "pandas", "streamlit", "plotly", "selenium", "SQLAlchemy", "python-dotenv", "pytest"]
def verificar() -> bool:
    """Exibe o diagnóstico do ambiente e devolve True se estiver tudo certo.
    O "-> bool" depois dos parênteses é uma anotação: avisa que a função
    devolve um valor do tipo bool (True ou False).
    """
    # sys.version é um texto como "3.14.3 (tags/...)"; .split() quebra o texto nos
    # espaços e [0] pega só o primeiro pedaço: "3.14.3".
    # sys.executable é o caminho do python.exe que está executando este arquivo.
    print(f"Python {sys.version.split()[0]} em {sys.executable}")
    # sys.version_info é uma tupla, por exemplo (3, 14, 3, ...). Comparar com (3, 12)
    # responde à pergunta "a versão é 3.12 ou mais nova?".
    ok = sys.version_info >= (3, 12)
    # Expressão condicional: escolhe "OK" se ok for True; caso contrário, o aviso.
    print("  versão do Python:", "OK" if ok else "ATENÇÃO: use 3.12 ou superior")
    # Dentro de um ambiente virtual, sys.prefix (pasta do Python em uso) é diferente
    # de sys.base_prefix (pasta do Python instalado no computador).
    # No PyCharm, isso confirma que o interpretador do projeto é o do .venv.
    dentro_do_venv = sys.prefix != sys.base_prefix
    print("  ambiente virtual ativo:", "sim" if dentro_do_venv else "NÃO — ative o .venv")
    # O laço "for" repete o bloco indentado uma vez para cada nome da lista PACOTES.
    for pacote in PACOTES:
        try:
```

exemplos/tutorial/m01_verificar_ambiente.py

```
            # metadata.version("pandas") devolve a versão instalada, por exemplo "3.0.6".
            # {pacote:<15} escreve o nome alinhado à esquerda ocupando 15 caracteres.
            print(f"  {pacote:<15} {metadata.version(pacote)}")
        except metadata.PackageNotFoundError:
            # Se o pacote não estiver instalado, o Python "lança" este erro.
            # O except captura o erro e o programa continua, em vez de parar.
            print(f"  {pacote:<15} NÃO INSTALADO")
            ok = False  # basta um pacote faltando para o ambiente não estar pronto
    # "and": só é True se as duas condições forem verdadeiras.
    return ok and dentro_do_venv
# ---------------------------------------------------------------------------
# PONTO DE ENTRADA
# Este bloco só executa quando o arquivo é rodado diretamente (botão ▶ do
# PyCharm ou "python arquivo.py"). Se outro arquivo fizer "import" deste,
# o bloco NÃO é executado — por isso os testes conseguem importar a função.
# ---------------------------------------------------------------------------
if __name__ == "__main__":
    print("Ambiente pronto!" if verificar() else "Corrija os itens acima antes de continuar.")
```

1.7 Explicação do código

- O texto entre aspas triplas no início é a **docstring do módulo**: explica o arquivo. O PyCharm mostra esse texto quando você pressiona Ctrl+Q sobre o nome do módulo.
- `sys.version_info >= (3, 12)` compara a versão do Python com 3.12.
- `sys.prefix != sys.base_prefix` só é verdadeiro **dentro** de um ambiente virtual — confirma que o PyCharm está usando o `.venv`.
- `metadata.version(pacote)` pergunta a versão de um pacote instalado; se ele não estiver instalado, ocorre `PackageNotFoundError`, tratado com `try/except`.
- O bloco `if __name__ == "__main__":` só roda quando o arquivo é executado diretamente — é nele que o PyCharm coloca o ▶ verde.

1.8 Resultado esperado

Na janela Run:

Saída real (21/10/2026)

```
Python 3.14.3 em C:\...\clima-alerta\.venv\Scripts\python.exe
  versão do Python: OK
  ambiente virtual ativo: sim
  requests        2.34.2
  pandas          3.0.6
  streamlit       1.66.1
  plotly          7.1.0
  selenium        4.51.0
  SQLAlchemy      2.1.6
  python-dotenv   1.2.4
  pytest          9.1.1
Ambiente pronto!
```

1.9 Verificação

O projeto possui testes automáticos para cada módulo do tutorial; eles testam **os seus arquivos** em `exemplos/tutorial/`. No PyCharm, abra `tests/test_tutorial.py` e clique no ▶ verde ao lado de `def test_m01_verifica_ambiente`. Ou, no Terminal do PyCharm:

Terminal do PyCharm — pasta raiz

```
pytest tests/test_tutorial.py -k m01 -v
```

Resultado esperado: o teste aparece com ✓ verde na janela Run (ou `PASSED` no Terminal).

1.10 Erros frequentes e soluções

Erro	Causa	Solução
Barra de status mostra "No interpreter"	Interpretador não configurado	Repita o passo 4
`ambiente virtual ativo: NÃO`	O projeto usa o Python do sistema	Settings > Python Interpreter: escolha o `.venv`
`ModuleNotFoundError: No module named 'streamlit'`	Dependências não instaladas no `.venv`	Repita o passo 5 (caminho A ou C)
"Install requirements" falha no meio	Internet instável ou antivírus	Use o caminho C no Terminal e leia a mensagem de erro
Terminal do PyCharm sem `(.venv)`	Ativação automática desligada	Settings > Tools > Terminal > marque "Activate virtualenv"; feche e abra o Terminal
"a execução de scripts foi desabilitada" (Windows)	Política do PowerShell	No Terminal: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` e responda S
Acentos aparecem como "Ã©"	Codificação diferente de UTF-8	Passo 6.1 (File Encodings)

1.11 Exercícios práticos

1. Abra **Settings > Project > Python Interpreter** e anote as versões de `pandas` e `selenium`.
2. Na janela Python Packages, pesquise o pacote `ruff` e veja a descrição e a versão instalada.
3. Abra o `.env` no editor e localize o intervalo de atualização. Qual é o valor padrão?
4. Pressione Shift duas vezes, digite `Terminal` e abra o Terminal por esse caminho.

1.12 Critérios de conclusão

- ☐ A barra de status mostra o interpretador do `.venv`.
- ☐ As dependências do `requirements.txt` estão instaladas.
- ☐ `m01_verificar_ambiente.py` mostra "Ambiente pronto!" e o teste `m01` passou.
- ☐ As configurações do passo 6 foram feitas.

1.13 Desafios de extensão

- Acrescente à lista `PACOTES` os pacotes `ruff` e `pytest-cov`.
- Faça o programa terminar com `sys.exit(1)` quando algo estiver errado e observe o "exit code" mostrado na janela Run.

Módulo 2 — Fundamentos de Python

2.1 Objetivo de aprendizagem

Usar variáveis, tipos, listas, dicionários, funções, condições (`if`), laços (`for`) e *f-strings* para processar dados de tempo, aproveitando os recursos do editor do PyCharm.

2.2 Pré-requisitos

Módulo 1 concluído (interpretador `.venv` configurado).

2.3 Conceitos

- **Variável:** nome que guarda um valor. Tipos básicos: `str` (texto), `int` (inteiro), `float` (decimal), `bool` (verdadeiro/falso). Em Python, o separador decimal é o **ponto** (`24.7`).
- **Lista** `[...]`: sequência ordenada de valores; o primeiro está na posição 0.
- **Dicionário** `{chave: valor}`: guarda dados por nome, como uma ficha.
- **Função** (`def`): bloco de código reutilizável que recebe parâmetros e devolve um resultado com `return`. As anotações (`: float`, `-> float`) indicam os tipos; o PyCharm usa essas anotações para avisar sobre erros.
- **Condição** (`if/elif/else`) e **laço** (`for`): controlam o fluxo do programa.
- **f-string** `f"{valor:.1f}"`: insere valores em textos com formatação.
- **Ajuda do editor:** o PyCharm completa nomes enquanto você digita (Enter ou Tab aceita a sugestão), sublinha erros em vermelho e mostra avisos em amarelo.

2.4 Arquivos a criar ou alterar

Arquivo	Ação
`exemplos/tutorial/m02_fundamentos.py`	Criar (botão direito na pasta > New > Python File)

2.5 Passo a passo no PyCharm

1. Crie o arquivo `m02_fundamentos` em `exemplos/tutorial`.
2. Digite o código da seção 2.6. Observe o **autocompletar**: ao digitar `temper`, o PyCharm sugere `temperatura` e `temperaturas`.
3. Pressione **Ctrl+Alt+L** para reformatar o código no padrão do Python (espaços e quebras de linha).
4. Execute com o ▶ ao lado de `if __name__ == "__main__":` (ou Ctrl+Shift+F10).
5. Experimente: posicione o cursor sobre `media` e pressione **Ctrl+Q** (documentação) e **Ctrl+B** (vai até a definição).

Equivalente no Terminal do PyCharm:

Terminal do PyCharm — pasta raiz

```
python exemplos/tutorial/m02_fundamentos.py
```

2.6 Código

exemplos/tutorial/m02_fundamentos.py

```
"""Módulo 2 — Fundamentos de Python usando dados de tempo.
O QUE ESTE ARQUIVO FAZ
    Guarda dados meteorológicos em variáveis, listas e dicionários; usa funções
    para converter temperaturas, classificá-las e calcular a média; percorre uma
    lista com um laço "for" e mostra tudo na tela com f-strings.
COMO EXECUTAR NO PYCHARM
```

exemplos/tutorial/m02_fundamentos.py

```
    Botão ▶ verde ao lado de  if __name__ == "__main__":  ou  Run 'm02_fundamentos'
    (Ctrl+Shift+F10). Terminal (Alt+F12), na pasta raiz:
        python exemplos/tutorial/m02_fundamentos.py
DICA DO PYCHARM
    Passe o mouse sobre um nome (por exemplo, "media") para ver a documentação.
    Ctrl+clique sobre um nome leva até onde ele foi definido.
"""
# ============================================================================
# 1. VARIÁVEIS E TIPOS
# Uma variável é um nome que guarda um valor. O sinal "=" ATRIBUI o valor.
# O Python descobre o tipo sozinho a partir do valor.
# ============================================================================
cidade = "Sapiranga"  # texto (str) — sempre entre aspas
temperatura = 24.7  # número decimal (float) — em Python o separador decimal é o PONTO
umidade = 58  # número inteiro (int)
chovendo = False  # valor lógico (bool): True (verdadeiro) ou False (falso)
# ============================================================================
# 2. LISTA
# Uma lista guarda vários valores em ordem, entre colchetes [ ].
# A primeira posição é a 0: temperaturas[0] vale 11.3.
# ============================================================================
temperaturas = [11.3, 14.0, 19.5, 24.7, 25.2, 21.0, 17.4]  # 7 medições ao longo do dia
# ============================================================================
# 3. DICIONÁRIO
# Um dicionário guarda pares "chave: valor" entre chaves { }. É como uma ficha:
# acessamos o valor pelo nome da chave, por exemplo leitura["cidade"].
# ============================================================================
leitura = {"cidade": cidade, "temperatura": temperatura, "umidade": umidade}
# ============================================================================
# 4. FUNÇÕES
# "def" cria uma função: um bloco de código com nome, que recebe valores
# (parâmetros) e devolve um resultado com "return".
# As anotações (": float" e "-> float") informam os tipos esperados; o PyCharm
# usa essas anotações para avisar quando passamos um valor de tipo errado.
# ============================================================================
def celsius_para_fahrenheit(celsius: float) -> float:
    """Converte graus Celsius em Fahrenheit: °F = °C × 9/5 + 32."""
    return celsius * 9 / 5 + 32  # a multiplicação e a divisão são feitas antes da soma
def classificar(temp: float) -> str:
    """Classifica a temperatura em "quente", "agradável" ou "frio".
    O Python testa as condições DE CIMA PARA BAIXO e executa somente o bloco da
    primeira condição verdadeira. Por isso 30 °C é "quente" (e não "agradável").
    """
    if temp >= 30:  # ">=" significa "maior ou igual a"
        return "quente"
    elif temp >= 18:  # "elif" = "senão, se..."; só é testado se o "if" for falso
        return "agradável"
    else:  # "else" = "em todos os outros casos"
        return "frio"
def media(valores: list[float]) -> float:
    """Calcula a média: soma de todos os valores dividida pela quantidade."""
    # sum() soma os elementos da lista; len() conta quantos elementos ela tem.
    return sum(valores) / len(valores)
# ============================================================================
# 5. PROGRAMA PRINCIPAL
# Executa somente quando o arquivo é rodado diretamente (botão ▶).
# ============================================================================
if __name__ == "__main__":
    # f-string: o "f" antes das aspas permite colocar variáveis entre { }.
    print(f"Cidade: {leitura['cidade']}")
    # {valor:.1f} formata o número com 1 casa decimal (76.46 vira 76.5).
    print(f"Agora: {temperatura} °C = {celsius_para_fahrenheit(temperatura):.1f} °F ({classificar(temperatura)})")
    # min() e max() encontram o menor e o maior valor da lista.
    print(f"Mínima: {min(temperaturas)} °C | Máxima: {max(temperaturas)} °C | Média: {media(temperaturas):.1f} °C")
    # 6. LAÇO FOR com zip(): percorre duas listas AO MESMO TEMPO, par a par.
    # Na 1ª volta, hora=0 e temp=11.3; na 2ª, hora=4 e temp=14.0; e assim por diante.
    # strict=True gera erro se as listas tiverem tamanhos diferentes (proteção contra enganos).
    for hora, temp in zip([0, 4, 8, 12, 14, 18, 22], temperaturas, strict=True):
        # {hora:02d} escreve o inteiro com 2 dígitos (0 vira "00");
        # {temp:5.1f} usa 5 caracteres e 1 casa decimal, alinhando a coluna.
        print(f"  {hora:02d}h -> {temp:5.1f} °C ({classificar(temp)})")
    # Expressão condicional em uma linha: "valor_se_verdadeiro if condição else valor_se_falso".
    print("Está chovendo?", "sim" if chovendo else "não")
```

2.7 Explicação do código

- As variáveis do início descrevem uma leitura; a lista `temperaturas` guarda sete medições do dia; o dicionário `leitura` agrupa dados por nome.
- `celsius_para_fahrenheit` aplica a fórmula °F = °C × 9/5 + 32.
- `classificar` mostra a ordem das condições: a primeira verdadeira é usada (30 °C é "quente", não "agradável").
- `media` usa `sum` e `len`; `min` e `max` encontram os extremos.
- `zip` percorre duas listas juntas; `{hora:02d}` escreve 0 como `00`.

2.8 Resultado esperado

Saída real (janela Run)

```
Cidade: Sapiranga
Agora: 24.7 °C = 76.5 °F (agradável)
Mínima: 11.3 °C | Máxima: 25.2 °C | Média: 19.0 °C
  00h ->  11.3 °C (frio)
  04h ->  14.0 °C (frio)
  08h ->  19.5 °C (agradável)
  ...
Está chovendo? não
```

2.9 Verificação

Abra `tests/test_tutorial.py` e clique no ▶ ao lado de `def test_m02_fundamentos`, ou no Terminal:

Terminal do PyCharm — pasta raiz

```
pytest tests/test_tutorial.py -k m02 -v
```

O teste confere, por exemplo, que `celsius_para_fahrenheit(100)` devolve 212 e que 5 °C é "frio".

2.10 Erros frequentes e soluções

Erro	Causa	Solução
`IndentationError`	Espaços inconsistentes	Use Tab (o PyCharm insere 4 espaços) e Ctrl+Alt+L
`NameError: name 'temperatura' is not defined`	Nome digitado diferente	O PyCharm sublinha em vermelho; use o autocompletar
`TypeError: can only concatenate str`	Soma de texto com número	Use f-string: `f"{temp} °C"`
`ZeroDivisionError` em `media([])`	Lista vazia	Verifique `if valores:` antes de dividir
Não aparece o ▶ verde	Falta o bloco `if __name__ == "__main__":` ou interpretador	Confira o código e a barra de status

2.11 Exercícios práticos

1. Crie a função `kmh_para_ms(v)` (divida por 3,6) e mostre o vento de 13,5 km/h em m/s.
2. Acrescente a faixa "muito quente" para temperaturas a partir de 35 °C. Onde o novo `if` deve ficar?
3. Conte quantas horas da lista foram classificadas como "frio".
4. Use **Refactor > Rename** (Shift+F6) para renomear a variável `leitura` para `ficha` e observe que todas as ocorrências mudam juntas.

2.12 Critérios de conclusão

- ☐ O programa executa e o teste `m02` passa.
- ☐ Você sabe explicar a diferença entre lista e dicionário.
- ☐ Você usou Ctrl+Q, Ctrl+B e Ctrl+Alt+L.

2.13 Desafios de extensão

- Leia a temperatura com `input()` (a digitação acontece na janela Run) e aceite vírgula decimal (`"24,7".replace(",", ".")`).
- Crie um dicionário com a previsão de 3 dias e mostre o dia mais quente.

Módulo 3 — Primeira dashboard com Streamlit

3.1 Objetivo de aprendizagem

Criar uma página web interativa com **Streamlit** (título, cartões, seletor, gráfico e tabela), executá-la pelo PyCharm com uma **Run Configuration** e explorar a dashboard de exemplo do projeto.

3.2 Pré-requisitos

Módulos 1 e 2 concluídos.

3.3 Conceitos

- **Dashboard:** painel visual que resume dados em cartões, gráficos e tabelas.
- **Streamlit:** biblioteca que transforma um script Python em página web. A cada interação, o script é **executado novamente de cima para baixo**.
- `streamlit run`**:** uma dashboard não é executada como um programa comum; ela é iniciada pelo comando `streamlit run arquivo.py`, que cria um pequeno servidor web local.
- **Run Configuration de módulo:** no PyCharm, executamos o **módulo** `streamlit` passando `run arquivo.py` como parâmetro — é o mesmo que digitar o comando no Terminal.
- **Dados simulados:** valores inventados para estudo — sempre identificados na tela.

3.4 Arquivos a criar ou alterar

Arquivo	Ação
`exemplos/tutorial/m03_primeira_dashboard.py`	Criar
Run Configuration "m03 - dashboard"	Criar no PyCharm
`exemplos/dashboard_python/`	Apenas executar e estudar (já existe)

3.5 Passo a passo no PyCharm

1. Crie `m03_primeira_dashboard` em `exemplos/tutorial` e digite o código da seção 3.6.
2. Crie a configuração de execução: menu **Run > Edit Configurations...** (ou a lista de configurações no topo da janela > Edit Configurations) > botão **+** > **Python**.
3. Preencha:
   - **Name:** `m03 - dashboard`
   - No campo que mostra **script**, abra a lista e escolha **module**; digite `streamlit`.
   - **Script parameters** (ou *Parameters*): `run exemplos/tutorial/m03_primeira_dashboard.py`
   - **Working directory:** a pasta raiz do projeto (já preenchida se você fez o passo 6 do módulo 1).
   - **Python interpreter:** o do projeto (`.venv`).
4. Clique em **OK**, escolha `m03 - dashboard` na lista do topo e clique em ▶ (Shift+F10).
5. A janela Run mostra o endereço `http://localhost:8501` e o navegador abre. Para parar, clique no quadrado vermelho ■ da janela Run (Ctrl+F2).

Equivalente no Terminal do PyCharm (Ctrl+C encerra):

Terminal do PyCharm — pasta raiz

```
streamlit run exemplos/tutorial/m03_primeira_dashboard.py
```

Depois, crie uma segunda configuração igual, com o nome `Dashboard de exemplo` e o parâmetro `run exemplos/dashboard_python/app.py`, e execute-a. (Dica: na janela Edit

Configurations, selecione `m03 - dashboard` e clique em **Copy Configuration** para duplicá-la.)

3.6 Código

exemplos/tutorial/m03_primeira_dashboard.py

```
"""Módulo 3 — Primeira dashboard com Streamlit (dados SIMULADOS digitados no código).
O QUE ESTE ARQUIVO FAZ
    Cria uma página web com título, aviso de dados simulados, três cartões de
    indicadores, um seletor, um gráfico de linha e uma tabela.
COMO EXECUTAR NO PYCHARM
    Uma dashboard Streamlit NÃO é executada com o botão ▶ comum: ela precisa do
    comando "streamlit run". Use uma das opções:
    a) Terminal do PyCharm (Alt+F12), na pasta raiz do projeto:
           streamlit run exemplos/tutorial/m03_primeira_dashboard.py
    b) Configuração de execução (Run Configuration), criada uma única vez:
           Run > Edit Configurations... > + > Python
           Name:               m03 - dashboard
           Script/Module:      troque "script" por "module" e digite  streamlit
           Script parameters:  run exemplos/tutorial/m03_primeira_dashboard.py
           Working directory:  pasta raiz do projeto
       Depois, escolha "m03 - dashboard" no topo da janela e clique em ▶.
    O navegador abre em http://localhost:8501. Para parar, clique no quadrado
    vermelho ■ da janela Run (ou Ctrl+C no Terminal).
COMO O STREAMLIT FUNCIONA
    A cada interação do usuário (por exemplo, escolher outra opção no seletor),
    o Streamlit executa este arquivo inteiro novamente, de cima para baixo.
"""
import pandas as pd  # Pandas: tabelas de dados (DataFrame). "pd" é o apelido usual.
import streamlit as st  # Streamlit: transforma o script em página web. "st" é o apelido usual.
# st.title escreve o título principal da página (letras grandes).
st.title("□□ Minha primeira dashboard")
# st.warning mostra uma faixa amarela. Usamos para deixar claro que os dados são inventados.
st.warning("Dados SIMULADOS, digitados no código para fins de estudo.")
# ------------------------------------------------------------------------------
# DADOS
# pd.DataFrame recebe um dicionário: cada CHAVE vira o nome de uma COLUNA e cada
# LISTA vira os valores dessa coluna. As três listas precisam ter o mesmo tamanho.
# ------------------------------------------------------------------------------
dados = pd.DataFrame(
    {
        "hora": ["06h", "09h", "12h", "15h", "18h", "21h"],  # coluna de textos
        "temperatura": [12.0, 18.5, 23.0, 25.5, 21.0, 16.5],  # em °C
        "chuva_mm": [0.0, 0.0, 0.2, 1.5, 3.0, 0.4],  # chuva em milímetros
    }
)
# ------------------------------------------------------------------------------
# CARTÕES (st.metric) lado a lado
# st.columns(3) divide a largura da página em 3 colunas e devolve uma "caixa"
# para cada uma. Escrever em col1 coloca o conteúdo na primeira coluna.
# ------------------------------------------------------------------------------
col1, col2, col3 = st.columns(3)
# dados["temperatura"] seleciona uma coluna; .max() devolve o maior valor dela.
col1.metric("Máxima", f"{dados['temperatura'].max():.1f} °C")
col2.metric("Mínima", f"{dados['temperatura'].min():.1f} °C")
# .sum() soma todos os valores da coluna de chuva.
col3.metric("Chuva total", f"{dados['chuva_mm'].sum():.1f} mm")
# ------------------------------------------------------------------------------
# CONTROLE INTERATIVO (widget)
# st.selectbox mostra uma lista suspensa e DEVOLVE a opção escolhida pelo usuário.
# Quando o usuário troca a opção, o script roda de novo e "variavel" muda.
# ------------------------------------------------------------------------------
variavel = st.selectbox("Variável do gráfico", ["temperatura", "chuva_mm"])
# Gráfico de linha: eixo x = coluna "hora"; eixo y = coluna escolhida no seletor.
st.line_chart(dados, x="hora", y=variavel)
# st.subheader escreve um subtítulo; st.dataframe mostra a tabela interativa.
```

exemplos/tutorial/m03_primeira_dashboard.py

```
st.subheader("Tabela")
st.dataframe(dados, hide_index=True)  # hide_index=True esconde a coluna de números 0, 1, 2...
```

3.7 Explicação do código

- `st.title` e `st.warning` escrevem o título e a faixa que identifica os dados como simulados.
- `pd.DataFrame({...})` cria a tabela a partir de um dicionário de listas (cada chave vira uma coluna).
- `st.columns(3)` divide a página em três colunas; `col.metric` desenha um cartão em cada uma.
- `st.selectbox` devolve a opção escolhida; ao trocar a opção, o Streamlit reexecuta o script e o gráfico muda.
- `st.line_chart` desenha o gráfico; `st.dataframe` mostra a tabela.

3.8 Resultado esperado

Uma página com o título "Minha primeira dashboard", a faixa amarela de dados simulados, três cartões (Máxima 25,5 °C; Mínima 12,0 °C; Chuva total 5,1 mm), um seletor, um gráfico de linha e a tabela. A dashboard de exemplo do projeto aparece assim:

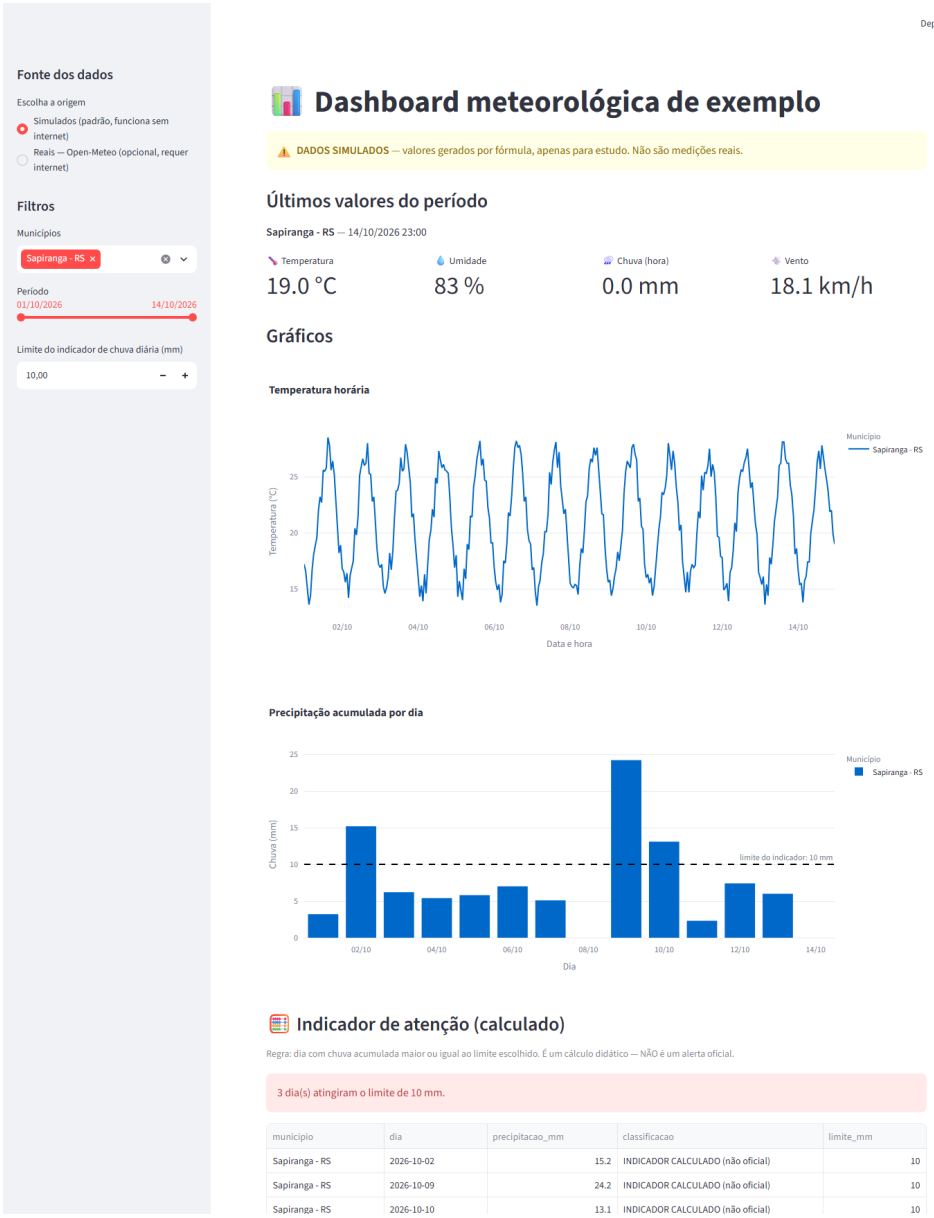


Figura 2 — Dashboard de exemplo independente (exemplos/dashboard_python/app.py) com dados simulados

3.9 Verificação

Abra `tests/test_tutorial.py` e execute `test_m03_dashboard` pelo ▶. Para a dashboard de exemplo, clique com o botão direito na pasta `exemplos/dashboard_python/tests` > **Run 'pytest in tests'**. No Terminal:

Terminal do PyCharm — pasta raiz

```
pytest tests/test_tutorial.py -k m03 -v
pytest exemplos/dashboard_python/tests -v
```

3.10 Erros frequentes e soluções

Erro	Causa	Solução
Executou com ▶ comum e apareceu "missing ScriptRunContext"	Dashboard executada como script	Use a Run Configuration de **module** `streamlit` ou o Terminal
`No module named streamlit`	Configuração usando outro interpretador	Na Run Configuration, escolha o interpretador `.venv`
"Port 8501 is already in use"	Outra dashboard aberta	Pare-a (■) ou acrescente `--server.port 8502` aos parâmetros
`File does not exist: exemplos/...`	Working directory errado	Defina a pasta raiz como Working directory
Página não atualiza após editar	Navegador com versão antiga	Clique em "Rerun" na página ou pressione R

3.11 Exercícios práticos

1. Acrescente um quarto cartão com a temperatura média.
2. Troque `st.line_chart` por `st.bar_chart` e compare.
3. Na dashboard de exemplo, selecione dois municípios e altere o limite do indicador para 5 mm. Quantos dias aparecem?
4. Na dashboard de exemplo, escolha "Reais — Open-Meteo". O que muda na faixa colorida?

3.12 Critérios de conclusão

- ☐ A configuração `m03 - dashboard` abre sua dashboard e o teste `m03` passa.
- ☐ Você executou a dashboard de exemplo e identificou dados simulados × reais.

3.13 Desafios de extensão

- Use `st.slider` para escolher o intervalo de horas exibido.
- Leia `exemplos/dashboard_python/dados/dados_simulados.csv` com `pd.read_csv` e mostre-o na sua página.

Módulo 4 — Fundamentos de HTML e seletores CSS

4.1 Objetivo de aprendizagem

Reconhecer a estrutura de uma página HTML, escrever **seletores CSS** para localizar elementos — a base do Web Scraping — e testá-los no navegador aberto a partir do PyCharm.

4.2 Pré-requisitos

Módulo 1. Navegador Google Chrome instalado.

4.3 Conceitos

- **HTML** descreve a estrutura da página com **elementos** (*tags*): `<h1>` título, `<p>` parágrafo, `<div>` bloco, `<span>` trecho, `<a>` link, `<ul>/<li>` lista.
- **Atributos** dão informações extras: `id` (único), `class` (pode repetir), `href` (destino do link), `data-*` (dados personalizados, como `data-uf="RS"`).
- **DOM:** a árvore de elementos que o navegador monta a partir do HTML.
- **Seletores CSS** apontam elementos:

Seletor	Encontra	Exemplo na página
`h1`	Elementos pela *tag*	O título
`#titulo`	Elemento pelo `id`	O título
`.cidade`	Elementos pela classe	Os 3 cartões
`div.cidade .temp`	`.temp` dentro de `div.cidade`	As 3 temperaturas
`.cidade.destaque`	Elemento com as duas classes	Porto Alegre
`[data-uf='SC']`	Pelo valor de um atributo	Florianópolis
`#lista-dias li`	Itens dentro da lista	3 dias
`a.mais`	Links com a classe `mais`	1 link

4.4 Arquivos a criar ou alterar

Arquivo	Ação
`exemplos/tutorial/m04_pagina.html`	Criar (botão direito na pasta > New > HTML File)

4.5 Passo a passo no PyCharm

1. Clique com o botão direito em `exemplos/tutorial` > **New > HTML File**, nome `m04_pagina`. Apague o conteúdo gerado e digite o código da seção 4.6. Repare no autocompletar de *tags* e no fechamento automático.
2. Com o arquivo aberto, passe o mouse no canto superior direito do editor: aparecem ícones de navegadores. Clique no ícone do **Chrome**. (Alternativa: botão direito no arquivo > **Open In > Browser > Chrome**.)
3. No Chrome, pressione **F12** (Ferramentas do desenvolvedor), aba **Console**, e teste os seletores abaixo, um por vez:

Console do Chrome (F12)

```
document.querySelectorAll(".cidade").length
document.querySelector("#titulo").textContent
document.querySelectorAll("div.cidade .temp")
document.querySelector("[data-uf='SC'] .nome").textContent
```

Console do Chrome (F12)

```
document.querySelector("a.mais").href
```

1. Na aba **Elements** do Chrome, clique em um elemento da página para ver o HTML correspondente.

4.6 Código

exemplos/tutorial/m04_pagina.html

```
<!doctype html>
<!--
  Módulo 4 — Página para praticar HTML e seletores CSS (DADOS FICTÍCIOS).
  COMO ABRIR NO PYCHARM
    Abra este arquivo no editor. No canto superior direito do editor aparecem
    ícones de navegadores: clique no ícone do Chrome para ver a página.
    (Alternativa: botão direito no arquivo > Open In > Browser > Chrome.)
    No Chrome, pressione F12 e use a aba Console para testar os seletores.
  ESTRUTURA DE UM ELEMENTO HTML
    <tag atributo="valor">conteúdo</tag>
    - tag: o tipo do elemento (h1 = título, p = parágrafo, div = bloco...)
    - atributos: informações extras (id, class, href, data-...)
-->
<html lang="pt-BR"> <!-- lang informa o idioma da página -->
<head> <!-- head: informações sobre a página (não aparecem no corpo) -->
  <meta charset="utf-8"> <!-- utf-8 permite acentos: ã, é, ç... -->
  <title>Módulo 4 — Página para praticar seletores CSS (dados fictícios)</title> <!-- texto da aba -->
  <style>
    /* CSS: regras de aparência. Cada regra usa um SELETOR para escolher elementos. */
    body { font-family: Arial, sans-serif; margin: 24px; }        /* seletor de tag: o corpo */
    .cidade { border: 1px solid #999; border-radius: 8px;          /* seletor de classe: ponto + nome */
              padding: 8px 12px; margin: 8px 0; width: 260px; }
    .destaque { background: #fff3c4; }                             /* fundo amarelo-claro */
  </style>
</head>
<body> <!-- body: o conteúdo visível -->
  <!-- id identifica UM único elemento da página. Seletor CSS: #titulo -->
  <h1 id="titulo">Boletim fictício do tempo</h1>
  <!-- Aviso obrigatório: os dados desta página são inventados -->
  <p class="aviso">Todos os dados desta página são FICTÍCIOS.</p>
  <!-- Cada cidade é um <div> com a classe "cidade" (seletor: .cidade ou div.cidade).
       data-uf é um atributo PERSONALIZADO que guarda a sigla do estado
       (seletor: [data-uf='RS']). -->
  <div class="cidade" data-uf="RS">
    <h2 class="nome">Sapiranga</h2>                                <!-- seletor: .nome -->
    <span class="temp">24,5 °C</span>                               <!-- seletor: .temp (vírgula decimal!) -->
    <a class="mais" href="https://open-meteo.com/">Fonte de exemplo</a> <!-- link: atributo href -->
  </div>
  <!-- Este cartão tem DUAS classes: "cidade" e "destaque" (seletor: .cidade.destaque) -->
  <div class="cidade destaque" data-uf="RS">
    <h2 class="nome">Porto Alegre</h2>
    <span class="temp">26,1 °C</span>
    <!-- Repare: este cartão NÃO tem o link <a class="mais">.
         No módulo 5 o programa precisa tratar esse elemento ausente. -->
  </div>
  <div class="cidade" data-uf="SC">
    <h2 class="nome">Florianópolis</h2>
    <span class="temp">22,0 °C</span>
  </div>
  <!-- Lista com marcadores: <ul> é a lista e cada <li> é um item (seletor: #lista-dias li) -->
  <ul id="lista-dias">
    <li>Segunda: 25 °C</li>
    <li>Terça: 27 °C</li>
    <li>Quarta: 19 °C</li>
  </ul>
</body>
</html>
```

4.7 Explicação do código

- Os comentários `<!-- ... -->` explicam cada parte e não aparecem na página.
- O `<head>` define a codificação `utf-8` (acentos) e as regras de aparência (CSS); cada regra começa com um seletor.
- Cada cidade é um `<div class="cidade">` com o atributo `data-uf`, um `<h2 class="nome">` e um `<span class="temp">`.
- Só Sapiranga tem o link `<a class="mais">` — isso permitirá praticar o tratamento de **elemento ausente** no módulo 5.
- O texto "Todos os dados desta página são FICTÍCIOS" identifica os dados.

4.8 Resultado esperado

No console: `3`, `'Boletim fictício do tempo'`, uma lista com 3 elementos, `'Florianópolis'` e `'https://open-meteo.com/'`.

4.9 Verificação

Execute o teste `test_arquivos_do_tutorial_existem` (▶ ao lado dele em `tests/test_tutorial.py`). Neste momento ele **ainda falhará**, apontando o primeiro arquivo que falta (dos módulos seguintes). O importante é que a mensagem **não** cite `m04_pagina.html`. Ele passará por completo ao final do módulo 12.

4.10 Erros frequentes e soluções

Erro	Causa	Solução
Acentos aparecem como "Ã©"	Arquivo sem UTF-8	Confira a codificação na barra de status do PyCharm (deve ser UTF-8)
`querySelector` devolve `null`	Seletor errado	Confira ponto (classe) × cerquilha (id)
`.cidade .destaque` não acha nada	Espaço significa "dentro de"	Use `.cidade.destaque` (sem espaço)
Ícone do Chrome não aparece	Navegador não detectado	Settings > Tools > Web Browsers and Preview: adicione o Chrome

4.11 Exercícios práticos

1. Escreva o seletor que encontra todos os nomes de cidades do RS.
2. Acrescente uma quarta cidade fictícia e confirme no console que `.cidade` passou a ter 4 elementos (o PyCharm recarrega a página ao salvar).
3. Abra `exemplos/selenium/pagina_meteorologica_demo.html` no Chrome pelo PyCharm e use F12 para descobrir a classe dos cartões.

4.12 Critérios de conclusão

- ☐ A página abre com acentos corretos a partir do PyCharm.
- ☐ Você testou pelo menos 5 seletores no console.

4.13 Desafios de extensão

- Use o seletor `li:nth-child(2)` para obter "Terça".
- Pesquise a diferença entre seletores CSS e XPath e escreva o XPath equivalente a `#titulo`.

Módulo 5 — Web Scraping com Selenium

5.1 Objetivo de aprendizagem

Controlar o navegador com **Selenium**: abrir uma página, esperar elementos com `WebDriverWait`, localizar por seletores CSS, extrair textos, atributos e links, tratar elementos ausentes, normalizar valores e fechar o navegador corretamente. Acompanhar a extração com o **depurador** do PyCharm e executar o scraper de demonstração do ClimaAlerta.

5.2 Pré-requisitos

Módulo 4. Google Chrome instalado. Internet na primeira execução (o **Selenium Manager** baixa o ChromeDriver).

5.3 Conceitos

- **Web Scraping:** extrair dados de páginas feitas para pessoas. Use apenas quando não houver API, respeite os termos de uso e o `robots.txt` e nunca contorne CAPTCHA ou login.
- **WebDriver:** "controle remoto" do navegador. `webdriver.Chrome()` abre o Chrome; `--headless=new` o executa sem janela.
- **Espera explícita** (`WebDriverWait(...).until(condição)`): aguarda até o elemento existir, no máximo N segundos.
- `find_element` (um elemento; erro se não existir) × `find_elements` (lista; vazia se não existir).
- **Normalização:** transformar "24,5 °C" (texto) em `24.5` (número).
- `try/finally`: garante que `navegador.quit()` rode mesmo se houver erro.
- **Ponto de parada (breakpoint):** marca onde o depurador deve pausar o programa.

5.4 Arquivos a criar ou alterar

Arquivo	Ação
`exemplos/tutorial/m05_selenium_basico.py`	Criar
`src/scraper/` e `scripts/executar_scraper_demo.py`	Estudar e executar (já existem)

5.5 Passo a passo no PyCharm

1. Crie `m05_selenium_basico` em `exemplos/tutorial` e digite o código da seção 5.6.
2. Execute com ▶. A primeira execução pode demorar porque o Selenium Manager baixa o driver.
3. **Depuração:** clique na margem esquerda (área cinza ao lado dos números de linha) na linha `for cartao in navegador.find_elements(...)` — aparece uma bolinha vermelha. Clique com o botão direito no editor > **Debug 'm05_selenium_basico'**. O programa pausa na linha marcada; pressione **F8** (Step Over) para avançar linha a linha e observe as variáveis `cartao`, `temp_texto` e `temperatura` na janela Debug. **F9** continua até o fim.
4. Para ver o navegador trabalhando, troque temporariamente `raspar()` por `raspar(headless=False)` no final do arquivo.
5. Execute o scraper completo do projeto: abra `scripts/executar_scraper_demo.py` e clique em ▶. Para ver a janela do Chrome, edite a configuração criada (Run > Edit Configurations > executar_scraper_demo) e preencha **Script parameters** com `--visivel`.

Equivalente no Terminal do PyCharm:

Terminal do PyCharm — pasta raiz

```
python exemplos/tutorial/m05_selenium_basico.py
```

Terminal do PyCharm — pasta raiz

```
python scripts/executar_scraper_demo.py
python scripts/executar_scraper_demo.py --visivel
```

5.6 Código

exemplos/tutorial/m05_selenium_basico.py

```
"""Módulo 5 — Web Scraping com Selenium na página local do módulo 4.
O QUE ESTE ARQUIVO FAZ
    Abre o Google Chrome controlado pelo programa, carrega a página
    m04_pagina.html, espera o título aparecer, percorre os cartões das cidades e
    extrai o nome, a UF (atributo), a temperatura (texto convertido em número) e
    o link, tratando o caso em que um elemento não existe. No fim, fecha o navegador.
COMO EXECUTAR NO PYCHARM
    Botão ▶ ao lado de  if __name__ == "__main__":  ou  Run 'm05_selenium_basico'.
    Terminal (Alt+F12), na pasta raiz:
        python exemplos/tutorial/m05_selenium_basico.py
PRÉ-REQUISITOS
    - Google Chrome instalado.
    - Internet na PRIMEIRA execução: o Selenium Manager baixa sozinho o
      "chromedriver" compatível com o seu Chrome.
DICA DO PYCHARM (depuração)
    Clique na margem esquerda da linha  for cartao in ...  para criar um ponto de
    parada (bolinha vermelha) e execute com  Debug 'm05_selenium_basico'
    (botão do inseto ). O programa pausa ali e você pode inspecionar as variáveis.
"""
from pathlib import Path  # Path representa caminhos de arquivos de forma independente do sistema
# Componentes do Selenium que vamos usar:
from selenium import webdriver  # controla o navegador
from selenium.common.exceptions import NoSuchElementException  # erro "elemento não encontrado"
from selenium.webdriver.chrome.options import Options  # opções de inicialização do Chrome
from selenium.webdriver.common.by import By  # tipos de localizador (By.ID, By.CSS_SELECTOR...)
from selenium.webdriver.support import expected_conditions as EC  # condições para esperar
from selenium.webdriver.support.ui import WebDriverWait  # espera explícita
# ---------------------------------------------------------------------------
# Endereço da página.
# Path(__file__) é o caminho DESTE arquivo; .with_name(...) troca o nome do arquivo,
# mantendo a mesma pasta; .resolve() gera o caminho completo; .as_uri() transforma
# em endereço que o navegador entende: file:///C:/.../m04_pagina.html
# Assim o programa funciona qualquer que seja a pasta de trabalho do PyCharm.
# ---------------------------------------------------------------------------
PAGINA = Path(__file__).with_name("m04_pagina.html").resolve().as_uri()
def texto_ou_none(elemento, seletor: str):
    """Devolve o texto do elemento filho que casa com o seletor, ou None se ele não existir.
    find_element LANÇA um erro (NoSuchElementException) quando não encontra nada.
    Capturamos esse erro para que um elemento ausente não interrompa a extração.
    """
    try:
        return elemento.find_element(By.CSS_SELECTOR, seletor).text  # .text = texto visível
    except NoSuchElementException:
        return None  # None significa "sem valor"
def raspar(headless: bool = True) -> list[dict]:
    """Abre a página, extrai os dados de cada cidade e devolve uma lista de dicionários.
    headless=True executa o Chrome sem abrir janela (mais rápido).
    Experimente chamar raspar(headless=False) para ver o navegador trabalhando.
    """
    opcoes = Options()
    if headless:
        opcoes.add_argument("--headless=new")  # modo sem janela
    # Abre o Chrome. O Selenium Manager localiza/baixa o driver automaticamente.
    navegador = webdriver.Chrome(options=opcoes)
    # try/finally: o bloco "finally" SEMPRE executa, mesmo se acontecer um erro no "try".
    # Isso garante que o navegador será fechado e não ficará aberto consumindo memória.
    try:
```

exemplos/tutorial/m05_selenium_basico.py

```
        navegador.get(PAGINA)  # carrega a página (como digitar o endereço no navegador)
        # ESPERA EXPLÍCITA: aguarda ATÉ 10 segundos o elemento com id="titulo" existir.
        # Se aparecer antes, segue imediatamente. Se não aparecer, gera TimeoutException.
        # Páginas reais costumam carregar partes do conteúdo depois de abrir;
        # sem esperar, o programa procuraria os elementos antes da hora.
        WebDriverWait(navegador, 10).until(EC.presence_of_element_located((By.ID, "titulo")))
        resultado = []  # lista vazia que vai receber um dicionário por cidade
        # find_elements (no plural) devolve uma LISTA com todos os elementos encontrados
        # (ou uma lista vazia). "div.cidade" = elementos <div> com a classe "cidade".
        for cartao in navegador.find_elements(By.CSS_SELECTOR, "div.cidade"):
            # Procuramos DENTRO do cartão (cartao.find_element...), e não na página inteira.
            temp_texto = texto_ou_none(cartao, ".temp")  # ex.: "24,5 °C"
            # NORMALIZAÇÃO: transformar o texto "24,5 °C" no número 24.5
            #   .split()           -> ["24,5", "°C"]  (quebra nos espaços)
            #   [0]                -> "24,5"          (primeiro pedaço)
            #   .replace(",", ".") -> "24.5"          (Python usa ponto decimal)
            #   float(...)         -> 24.5            (texto vira número)
            # Se o texto não existir (None), a temperatura também fica None.
            temperatura = float(temp_texto.split()[0].replace(",", ".")) if temp_texto else None
            # Para o link usamos find_elements: se não houver link, a lista vem vazia
            # e nenhum erro é lançado (outra forma de tratar elementos ausentes).
            link = cartao.find_elements(By.CSS_SELECTOR, "a.mais")
            resultado.append(  # .append acrescenta um item ao final da lista
                {
                    "cidade": texto_ou_none(cartao, ".nome"),
                    "uf": cartao.get_attribute("data-uf"),  # lê o valor de um ATRIBUTO do elemento
                    "temperatura": temperatura,
                    # link[0] é o primeiro elemento da lista; get_attribute("href") lê o endereço.
                    "link": link[0].get_attribute("href") if link else None,
                }
            )
        return resultado
    finally:
        navegador.quit()  # fecha o navegador e encerra o chromedriver
if __name__ == "__main__":
    # Mostra cada dicionário em uma linha.
    for linha in raspar():
        print(linha)
```

Seletores usados pelo scraper completo (centralizados em `src/scraper/selectors.py`):

Constante	Seletor CSS	Finalidade
PAGE_READY	#painel-cidades[data-status='carregado']	Indica que o conteúdo dinâmico foi carregado
DEMO_NOTICE	#aviso-dados-ficticios	Faixa que identifica os dados como fictícios
UPDATED_AT	#atualizado-em	Data/hora da última atualização (atributo datetime)
CITY_CARD	article.cartao-cidade	Cartão de cada cidade (um por cidade)
CITY_NAME	.cidade-nome	Nome da cidade dentro do cartão
CITY_STATE	.cidade-uf	Sigla da UF dentro do cartão
TEMPERATURE	.temperatura	Texto da temperatura, ex.: '24,5 °C'
HUMIDITY	.umidade	Texto da umidade, ex.: 'Umidade: 57 %'
WIND	.vento	Texto do vento, ex.: 'Vento: 13,5 km/h (SE)'
CONDITION	.condicao	Descrição da condição do tempo
DETAILS_LINK	a.link-detalhes	Link para a página de detalhes (atributo href)
FORECAST_ROWS	#tabela-previsao tbody tr	Linhas da tabela de previsão
FORECAST_CELLS	td	Células de uma linha da tabela

5.7 Explicação do código

- `Path(...).resolve().as_uri()` transforma o caminho do arquivo em endereço `file:///…`, que o navegador entende.
- `WebDriverWait(navegador, 10).until(...)` espera o título por até 10 s.
- `find_elements(By.CSS_SELECTOR, "div.cidade")` devolve a lista de cartões; para cada um, buscamos elementos **dentro** do cartão.
- `texto_ou_none` captura `NoSuchElementException` e devolve `None`: um elemento ausente não interrompe a extração.
- `get_attribute("data-uf")` lê um atributo; `get_attribute("href")` lê o link.
- A expressão com `split`, `replace` e `float` normaliza "24,5 °C" para 24.5.
- No scraper completo, `src/scraper/normalizer.py` faz o mesmo de forma mais robusta e `weather_scraper.py` registra os campos ausentes em `missing_fields`.

5.8 Resultado esperado

Saída real — m05_selenium_basico.py

```
{'cidade': 'Sapiranga', 'uf': 'RS', 'temperatura': 24.5, 'link': 'https://open-meteo.com/'}
{'cidade': 'Porto Alegre', 'uf': 'RS', 'temperatura': 26.1, 'link': None}
{'cidade': 'Florianópolis', 'uf': 'SC', 'temperatura': 22.0, 'link': None}
```

Saída real — scripts/executar_scraper_demo.py (trecho)

```
Dados fictícios identificados na página: sim
- Sapiranga/RS: temp=24.5 umid=57.0 vento=13.5 SE cond=Parcialmente nublado
- Cidade Exemplo/SC: temp=19.8 umid=None vento=7.2 S cond=Chuva fraca  [ausentes: humidity]
- Vila Modelo/PR: temp=None umid=88.0 vento=4.0 N cond=Nevoeiro  [ausentes: temperature]
CSV gerado: ...\data\scraper_demo_resultado.csv
```

5.9 Verificação

Execute `test_m05_selenium` pelo ▶ em `tests/test_tutorial.py` e, em seguida, clique com o botão direito em `tests/test_scraper_selenium.py` > **Run 'pytest in test_scraper_selenium.py'**. No Terminal:

Terminal do PyCharm — pasta raiz

```
pytest tests/test_tutorial.py -k m05 -v
pytest tests/test_scraper_selenium.py tests/test_scraper_normalizer.py -v
```

5.10 Erros frequentes e soluções

Erro	Causa	Solução
`SessionNotCreatedException`	Chrome ausente ou incompatível	Instale/atualize o Chrome; execute com internet uma vez
`TimeoutException`	Elemento não apareceu no prazo	Confira o seletor; aumente o tempo de espera
`NoSuchElementException`	`find_element` sem o elemento	Use `find_elements` ou `try/except`
Chrome continua aberto após parar no depurador	O `finally` não chegou a executar	Pressione F9 até o fim, ou feche o Chrome manualmente
`ValueError: could not convert string to float`	Texto como "—"	Verifique valores ausentes antes de converter

5.11 Exercícios práticos

1. Extraia também os 3 dias de `#lista-dias li` e converta "25 °C" em número.
2. No depurador, use **Evaluate Expression** (Alt+F8) para executar `cartao.text` enquanto o programa está pausado.
3. Na página de demonstração do projeto, qual cidade não tem o elemento de umidade? Confirme abrindo `data/scraper_demo_resultado.csv` no PyCharm.

5.12 Critérios de conclusão

- ☐ Seu scraper extrai as 3 cidades e o teste `m05` passa.
- ☐ Você usou um ponto de parada e o F8 no depurador.
- ☐ Você sabe explicar por que a espera explícita é necessária.

5.13 Desafios de extensão

- Modifique a página do módulo 4 para inserir as cidades com JavaScript após 2 segundos (`setTimeout`) e ajuste a espera.
- Salve o resultado em CSV com Pandas.

Módulo 6 — Consumo de APIs

6.1 Objetivo de aprendizagem

Fazer requisições HTTP a uma **API REST** com a biblioteca `requests`, enviar parâmetros, interpretar a resposta **JSON**, tratar erros de rede e usar *timeout*.

6.2 Pré-requisitos

Módulo 2. Internet.

6.3 Conceitos

- **API** (*Application Programming Interface*): um "balcão de atendimento" para programas. Enviamos um pedido (requisição) e recebemos dados estruturados.
- **HTTP GET** com **parâmetros**: `https://api.open-meteo.com/v1/forecast?latitude=-29.63&longitude=-51.00&current=temperature_2m`.
- **JSON:** formato de texto com objetos `{}` e listas `[]`; em Python vira dicionários e listas.
- **Código de status:** 200 = sucesso; 4xx = erro no pedido; 5xx = erro no servidor. `raise_for_status()` transforma 4xx/5xx em exceção.
- **Timeout:** tempo máximo de espera. Sem ele, o programa pode travar para sempre.
- **Atribuição:** os dados da Open-Meteo têm licença CC BY 4.0 — é obrigatório citar a fonte.
- **API × Web Scraping:** quando existe API, ela é preferível. Por isso o ClimaAlerta usa a API do INMET para os avisos e o Selenium só na página de demonstração.
- **Injeção de dependência:** passar "quem faz a requisição" como parâmetro (`http=requests`) permite que os testes usem uma resposta gravada, sem internet.

6.4 Arquivos a criar ou alterar

Arquivo	Ação
`exemplos/tutorial/m06_api_open_meteo.py`	Criar

6.5 Passo a passo no PyCharm

1. Veja a resposta "crua" da API no navegador: clique no link consulta à API Open-Meteo para Sapiranga. O Chrome mostra o JSON (marque "Estilo de formatação" para ler melhor). O endereço completo é a junção do endereço base `https://api.open-meteo.com/v1/forecast` com os parâmetros `latitude=-29.63806`, `longitude=-51.00694`, `current=temperature_2m,relative_humidity_2m,wind_speed_10m` e `timezone=America/Sao_Paulo`, separados por `&` após o sinal `?`.

1. Crie `m06_api_open_meteo` em `exemplos/tutorial` e digite o código da seção 6.6. Ao digitar `requests.`, o PyCharm lista as funções disponíveis; Ctrl+Q sobre `get` mostra a documentação.
2. Execute com ▶.
3. **Explore com o depurador:** marque um ponto de parada na linha `atual = dados["current"]`, execute com **Debug** e, na janela Debug, expanda a variável `dados` para ver a estrutura do JSON como dicionário.

Equivalente no Terminal do PyCharm:

Terminal do PyCharm — pasta raiz

```
python exemplos/tutorial/m06_api_open_meteo.py
```

6.6 Código

exemplos/tutorial/m06_api_open_meteo.py

```
"""Módulo 6 — Consumo da API Open-Meteo com a biblioteca requests.
O QUE ESTE ARQUIVO FAZ
    Envia uma requisição HTTP para a API de previsão da Open-Meteo pedindo as
    condições atuais de Sapiranga - RS, recebe a resposta em JSON, extrai
    temperatura, umidade e vento e mostra na tela. Trata falhas de rede.
COMO EXECUTAR NO PYCHARM
    Botão ▶ ao lado de  if __name__ == "__main__":  ou  Run 'm06_api_open_meteo'.
    Terminal (Alt+F12), na pasta raiz:
        python exemplos/tutorial/m06_api_open_meteo.py
    Precisa de internet. Dados: Open-Meteo.com (licença CC BY 4.0 — cite a fonte).
COMO FUNCIONA UMA API (resumo)
    1. Montamos um ENDEREÇO com PARÂMETROS:  ...forecast?latitude=-29.6&longitude=-51.0...
    2. Enviamos uma requisição GET (pedido de leitura).
    3. O servidor responde com um CÓDIGO DE STATUS (200 = sucesso) e um texto JSON.
    4. Convertemos o JSON em dicionários e listas do Python.
"""
import requests  # biblioteca para fazer requisições HTTP
# Endereço base da API (sem os parâmetros). Constante: não muda durante o programa.
URL = "https://api.open-meteo.com/v1/forecast"
def buscar_condicoes(latitude: float, longitude: float, http=requests) -> dict:
    """Pede as condições atuais para as coordenadas e devolve um dicionário simples.
    Parâmetros:
        latitude, longitude: coordenadas do município (graus decimais).
        http: quem faz a requisição. No uso normal é a própria biblioteca requests.
              Nos testes, passamos um objeto FALSO que devolve uma resposta gravada,
              assim o teste funciona sem internet (técnica: injeção de dependência).
    Devolve:
        {"horario": ..., "temperatura": ..., "umidade": ..., "vento": ...}
    """
    # Os parâmetros ficam em um dicionário; o requests monta o "?chave=valor&..." sozinho.
    parametros = {
        "latitude": latitude,
        "longitude": longitude,
        # "current" lista as variáveis desejadas, separadas por vírgula (nomes da documentação).
        "current": "temperature_2m,relative_humidity_2m,wind_speed_10m",
        "timezone": "America/Sao_Paulo",  # horários no fuso de Brasília
    }
    # GET com timeout: se o servidor não responder em 15 segundos, desiste com erro.
    # NUNCA faça requisições sem timeout — o programa poderia ficar travado para sempre.
    resposta = http.get(URL, params=parametros, timeout=15)
    # raise_for_status() lança um erro se o status for 4xx (erro no pedido)
    # ou 5xx (erro no servidor). Com status 200, não faz nada.
    resposta.raise_for_status()
    # .json() converte o texto JSON da resposta em dicionário Python.
    dados = resposta.json()
    # A resposta tem a forma {"current": {"time": ..., "temperature_2m": ..., ...}, ...}
    atual = dados["current"]  # acessamos o dicionário interno pela chave "current"
    # Devolvemos um dicionário novo, com nomes em português.
    return {
        "horario": atual["time"],
        "temperatura": atual["temperature_2m"],  # °C
        "umidade": atual["relative_humidity_2m"],  # %
        "vento": atual["wind_speed_10m"],  # km/h
    }
if __name__ == "__main__":
    try:
        # Coordenadas de Sapiranga - RS (obtidas na geocodificação da Open-Meteo).
        condicoes = buscar_condicoes(-29.63806, -51.00694)
        print("Condições atuais em Sapiranga (Open-Meteo):")
        # .items() percorre os pares (chave, valor) do dicionário.
        for chave, valor in condicoes.items():
            print(f"  {chave}: {valor}")
```

exemplos/tutorial/m06_api_open_meteo.py

```
    except requests.RequestException as erro:
        # RequestException agrupa falhas de rede, timeout e erros HTTP.
        # Em vez de "quebrar", o programa mostra uma mensagem amigável.
        print(f"Não foi possível consultar a API: {erro}")
```

6.7 Explicação do código

- `parametros` é um dicionário; o `requests` monta o endereço com `?latitude=...&longitude=...` automaticamente.
- `timeout=15` limita a espera a 15 segundos.
- `resposta.json()` converte o texto JSON em dicionário; `dados["current"]["temperature_2m"]` navega nele.
- O parâmetro `http=requests` permite que o teste troque a internet por um objeto falso. O ClimaAlerta completo faz o mesmo em `src/services/context.py`.
- `except requests.RequestException` captura falhas de rede, timeout e erros HTTP, mostrando uma mensagem amigável.

6.8 Resultado esperado

Saída real (21/10/2026 15:45)

```
Condições atuais em Sapiranga (Open-Meteo):
  horario: 2026-10-21T15:45
  temperatura: 23.6
  umidade: 62
  vento: 13.5
```

Sem internet: `Não foi possível consultar a API: ...` (o programa não "quebra").

6.9 Verificação

Execute `test_m06_api_com_resposta_real_gravada` pelo ▶ em `tests/test_tutorial.py`. No Terminal:

Terminal do PyCharm — pasta raiz

```
pytest tests/test_tutorial.py -k m06 -v
pytest tests/test_open_meteo.py tests/test_http_client.py -v
```

O teste do tutorial usa a resposta real gravada em `tests/fixtures/open_meteo_forecast_sapiranga.json` (abra esse arquivo no PyCharm para ver o JSON completo) e confere se o `timeout` foi informado.

6.10 Erros frequentes e soluções

Erro	Causa	Solução
`KeyError: 'current'`	Faltou o parâmetro `current`	Confira o dicionário de parâmetros
`HTTPError: 400`	Parâmetro inválido (ex.: latitude 200)	Leia o campo `reason` no JSON de resposta
`ConnectionError`	Sem internet ou bloqueio de rede	Teste o endereço no navegador; fale com o técnico
Programa travado	Requisição sem `timeout`	Sempre use `timeout=`
`requests` sublinhado em vermelho	Pacote não instalado no `.venv`	Passe o mouse e clique em "Install package requests", ou repita o passo 5 do módulo 1

6.11 Exercícios práticos

1. Acrescente a variável `apparent_temperature` (sensação térmica) aos parâmetros e ao resultado.
2. Consulte Porto Alegre (−30,03283; −51,23019) e compare com Sapiranga.
3. Peça `daily=temperature_2m_max,temperature_2m_min` e mostre a máxima e a mínima de hoje.

6.12 Critérios de conclusão

- ☐ O programa mostra os dados atuais e o teste `m06` passa.
- ☐ Você inspecionou o JSON no depurador.
- ☐ Você sabe explicar o que é JSON, código de status e timeout.

6.13 Desafios de extensão

- Use a API de geocodificação (`https://geocoding-api.open-meteo.com/v1/search?name=Gramado&countryCode=BR`) para descobrir as coordenadas de outra cidade e depois consultar o tempo dela.
- Compare seu código com `src/integrations/open_meteo.py` (Ctrl+Shift+N abre arquivos pelo nome): o que o ClimaAlerta valida a mais?

Módulo 7 — Tratamento de dados com Pandas

7.1 Objetivo de aprendizagem

Carregar dados JSON em um **DataFrame**, converter datas, filtrar linhas, calcular estatísticas, agrupar por dia, exportar para CSV e inspecionar tabelas no depurador do PyCharm.

7.2 Pré-requisitos

Módulos 2 e 6.

7.3 Conceitos

- **DataFrame:** tabela do Pandas; cada coluna é uma **Series**.
- `pd.to_datetime`: converte texto ("2026-10-21T14:00") em data/hora, permitindo extrair dia, hora etc. (`.dt.date`).
- **Filtro:** `df[df["temperatura"] >= 25]` mantém só as linhas que atendem à condição.
- **Agrupamento:** `groupby("dia").agg(...)` calcula mínimo, máximo, soma por grupo.
- **CSV brasileiro:** separador `;` e vírgula decimal para abrir corretamente no Excel em português.
- **Dados reais offline:** usamos uma resposta real da Open-Meteo gravada em 21/10/2026; assim todos obtêm o mesmo resultado.

7.4 Arquivos a criar ou alterar

Arquivo	Ação
`exemplos/tutorial/m07_pandas.py`	Criar
`exemplos/tutorial/resumo_diario.csv`	Gerado pelo programa

7.5 Passo a passo no PyCharm

1. Crie `m07_pandas` em `exemplos/tutorial` e digite o código da seção 7.6.
2. Execute com ▶ e leia a saída na janela Run.
3. **Ver a tabela inteira:** marque um ponto de parada na linha `resumo = resumo_por_dia(horario)` e execute com Debug. Na janela Debug, clique com o botão direito na variável `horario` > **View as DataFrame**: a tabela de 168 linhas aparece em uma grade, com rolagem e ordenação.
4. Depois de executar, o arquivo `resumo_diario.csv` aparece na janela Project. Abra-o no PyCharm (visualização de texto) e também no Excel ou LibreOffice Calc (botão direito > **Open In > Explorer** para localizar o arquivo).

Equivalente no Terminal do PyCharm:

Terminal do PyCharm — pasta raiz

```
python exemplos/tutorial/m07_pandas.py
```

7.6 Código

exemplos/tutorial/m07_pandas.py

```
"""Módulo 7 — Tratamento de dados com Pandas.
O QUE ESTE ARQUIVO FAZ
    Lê uma resposta REAL da Open-Meteo gravada em 21/10/2026 (arquivo
    tests/fixtures/open_meteo_forecast_sapiranga.json), monta uma tabela
    (DataFrame) com 168 horas de previsão, calcula estatísticas, filtra as horas
    mais quentes, resume os dados por dia e salva o resumo em CSV.
    Como os dados estão gravados, funciona sem internet e todos obtêm o mesmo resultado.
COMO EXECUTAR NO PYCHARM
    Botão ▶ ao lado de  if __name__ == "__main__":  ou  Run 'm07_pandas'.
    Terminal (Alt+F12), na pasta raiz:
        python exemplos/tutorial/m07_pandas.py
DICA DO PYCHARM
    Em modo Debug, ao parar em um ponto de parada, clique com o botão direito em
    uma variável DataFrame na janela Debug e escolha "View as DataFrame" para ver
    a tabela inteira em uma grade.
"""
import json  # lê textos no formato JSON
from pathlib import Path  # caminhos de arquivos
import pandas as pd  # tabelas de dados
# ---------------------------------------------------------------------------
# CAMINHOS
# parents[2] sobe duas pastas a partir deste arquivo:
#   exemplos/tutorial/m07_pandas.py -> exemplos/tutorial -> exemplos -> (raiz do projeto)
# Assim o caminho funciona mesmo que a pasta de trabalho do PyCharm seja outra.
# ---------------------------------------------------------------------------
RAIZ = Path(__file__).resolve().parents[2]
ARQUIVO = RAIZ / "tests" / "fixtures" / "open_meteo_forecast_sapiranga.json"  # "/" junta partes do caminho
SAIDA = Path(__file__).with_name("resumo_diario.csv")  # o CSV é salvo nesta mesma pasta
def carregar_horario(caminho: Path = ARQUIVO) -> pd.DataFrame:
    """Lê o JSON e devolve um DataFrame com uma linha por hora."""
    # read_text lê o arquivo como texto; json.loads converte o texto em dicionário.
    dados = json.loads(caminho.read_text(encoding="utf-8"))
    # dados["hourly"] é um dicionário de LISTAS: {"time": [...], "temperature_2m": [...], ...}
    # O DataFrame transforma cada lista em uma coluna (168 linhas = 7 dias x 24 horas).
    df = pd.DataFrame(dados["hourly"])
    # A coluna "time" vem como texto ("2026-10-21T14:00"). to_datetime converte em
    # data/hora de verdade, permitindo extrair o dia, a hora etc.
    df["time"] = pd.to_datetime(df["time"])
    # rename troca os nomes das colunas: {"nome_antigo": "nome_novo"}.
    return df.rename(columns={"time": "horario", "temperature_2m": "temperatura", "precipitation": "chuva_mm"})
def resumo_por_dia(df: pd.DataFrame) -> pd.DataFrame:
    """Agrupa as horas por dia e calcula mínima, máxima e chuva total de cada dia."""
    # assign cria uma coluna nova "dia" com apenas a data (sem a hora) de cada linha.
    # .dt dá acesso às partes de uma data/hora; .date extrai a data.
    df = df.assign(dia=df["horario"].dt.date)
    return (
        df.groupby("dia")  # 1) separa as linhas em grupos, um grupo por dia
        .agg(  # 2) calcula um valor por grupo: nova_coluna=("coluna_original", "função")
            minima=("temperatura", "min"),
            maxima=("temperatura", "max"),
            chuva_total=("chuva_mm", "sum"),
        )
        .round(1)  # 3) arredonda para 1 casa decimal
        .reset_index()  # 4) transforma "dia" (que virou índice) de volta em coluna
    )
if __name__ == "__main__":
    horario = carregar_horario()
    # head(5) mostra só as 5 primeiras linhas; [[...]] escolhe as colunas exibidas.
    print(horario[["horario", "temperatura", "chuva_mm"]].head(5))
    # len(df) = número de linhas; .mean() = média da coluna.
    print(f"\n{len(horario)} linhas; temperatura média: {horario['temperatura'].mean():.1f} °C")
    # FILTRO: horario["temperatura"] >= 25 gera uma coluna de True/False;
    # usada entre colchetes, mantém só as linhas em que a condição é True.
    quentes = horario[horario["temperatura"] >= 25]
    print(f"Horas com 25 °C ou mais: {len(quentes)}")
    resumo = resumo_por_dia(horario)
    print("\nResumo por dia:")
    print(resumo)
```

exemplos/tutorial/m07_pandas.py

```
    # CSV no padrão brasileiro: separador ";" e vírgula decimal, para abrir no Excel/LibreOffice.
    # index=False não grava a coluna de índice; utf-8-sig faz o Excel reconhecer os acentos.
    resumo.to_csv(SAIDA, sep=";", decimal=",", index=False, encoding="utf-8-sig")
    print(f"\nArquivo salvo: {SAIDA}")
```

7.7 Explicação do código

- `Path(__file__).resolve().parents[2]` sobe duas pastas a partir do arquivo até a raiz do projeto, para encontrar `tests/fixtures`.
- `pd.DataFrame(dados["hourly"])` transforma o dicionário de listas da API em tabela (168 linhas = 7 dias × 24 h).
- `rename` dá nomes em português às colunas.
- `assign(dia=...)` cria uma coluna com a data; `groupby("dia").agg(...)` calcula mínima, máxima e chuva total de cada dia.
- `to_csv(..., sep=";", decimal=",", encoding="utf-8-sig")` grava no padrão usado pelo ClimaAlerta (regra RN011).

7.8 Resultado esperado

Saída real (trecho)

```
168 linhas; temperatura média: 18.9 °C
Horas com 25 °C ou mais: 18
Resumo por dia:
          dia  minima  maxima  chuva_total
0  2026-10-21    11.5    25.1          0.0
1  2026-10-22    14.8    27.5          0.9
...
4  2026-10-25    14.5    17.9         25.2
```

7.9 Verificação

Execute `test_m07_pandas` pelo ▶ em `tests/test_tutorial.py`, ou no Terminal:

Terminal do PyCharm — pasta raiz

```
pytest tests/test_tutorial.py -k m07 -v
```

Abra `resumo_diario.csv` na planilha: os números devem aparecer em colunas separadas e com vírgula decimal.

7.10 Erros frequentes e soluções

Erro	Causa	Solução
`FileNotFoundError`	Arquivo de dados movido	Confira `tests/fixtures` na janela Project
`AttributeError: Can only use .dt accessor`	Coluna ainda é texto	Use `pd.to_datetime` antes
CSV abre tudo em uma coluna	Separador diferente do esperado	Use `sep=";"`
"View as DataFrame" não aparece	O programa não está pausado	Execute com Debug e um ponto de parada

7.11 Exercícios práticos

1. Calcule a umidade média por dia (coluna `relative_humidity_2m`).
2. Descubra a hora mais quente de toda a semana (`idxmax`).
3. Filtre as horas com probabilidade de chuva ≥ 70 % (`precipitation_probability`).

7.12 Critérios de conclusão

- ☐ O resumo por dia aparece e o teste `m07` passa.
- ☐ Você visualizou um DataFrame com "View as DataFrame".
- ☐ O CSV abre corretamente na planilha.

7.13 Desafios de extensão

- Faça um gráfico com `df.plot(x="horario", y="temperatura")` (requer matplotlib; o PyCharm mostra o gráfico na janela **Plots**).
- Compare seu resumo com a previsão diária oficial da API (`daily`): por que a máxima pode diferir um pouco?

Módulo 8 — Modelagem e persistência com SQLite e SQLAlchemy

8.1 Objetivo de aprendizagem

Modelar tabelas como classes Python (ORM), criar o banco SQLite, inserir e consultar dados, usar chaves primárias, estrangeiras e restrições de unicidade para impedir duplicações.

8.2 Pré-requisitos

Módulos 2 e 7.

8.3 Conceitos

- **Banco de dados relacional:** dados em **tabelas** (linhas e colunas) relacionadas entre si.
- **SQLite:** banco em um único arquivo, sem servidor.
- **ORM (SQLAlchemy):** cada tabela é uma classe; cada linha, um objeto. O SQLAlchemy gera o SQL.
- **Chave primária (PK):** identifica cada linha (`id`). **Chave estrangeira (FK):** aponta para a PK de outra tabela (`municipio_id`).
- **Relacionamento 1:N:** um município tem muitas observações.
- **Restrição UNIQUE:** impede repetir valores (aqui, a mesma hora para o mesmo município).
- **Sessão e transação:** a sessão agrupa operações; `commit` grava, `rollback` desfaz.
- **Idempotência:** executar o programa duas vezes não duplica os dados.

8.4 Arquivos a criar ou alterar

Arquivo	Ação
`exemplos/tutorial/m08_banco.py`	Criar
`exemplos/tutorial/tutorial.db`	Criado pelo programa

8.5 Passo a passo no PyCharm

1. Crie `m08_banco` em `exemplos/tutorial` e digite o código da seção 8.6. Observe que o PyCharm completa os parâmetros de `mapped_column` e mostra a assinatura com Ctrl+P.
2. Execute com ▶ **duas vezes** e compare: a segunda execução não deve duplicar dados.
3. O arquivo `tutorial.db` aparece na janela Project. Para ver as tabelas:
   - com a licença Pro/educacional: arraste `tutorial.db` para a janela **Database** (View > Tool Windows > Database) e expanda as tabelas;
   - na versão gratuita: abra o arquivo com o programa gratuito **DB Browser for SQLite** (sqlitebrowser.org).

Equivalente no Terminal do PyCharm:

Terminal do PyCharm — pasta raiz

```
python exemplos/tutorial/m08_banco.py
python exemplos/tutorial/m08_banco.py
```

8.6 Código

exemplos/tutorial/m08_banco.py

```
"""Módulo 8 — Modelagem e persistência com SQLite e SQLAlchemy.
O QUE ESTE ARQUIVO FAZ
    Define duas tabelas como classes Python (Municipio e Observacao), cria o banco
    SQLite exemplos/tutorial/tutorial.db, grava um município com quatro
    observações (sem duplicar se executado de novo), faz uma consulta com filtro
    e demonstra que o banco RECUSA uma observação duplicada.
COMO EXECUTAR NO PYCHARM
    Botão ▶ ao lado de  if __name__ == "__main__":  ou  Run 'm08_banco'.
    Execute DUAS vezes e compare: a segunda execução não duplica os dados.
    Terminal (Alt+F12), na pasta raiz:
        python exemplos/tutorial/m08_banco.py
VENDO O BANCO
    O arquivo tutorial.db aparece na janela Project depois da primeira execução.
    Para abrir as tabelas, use o programa gratuito "DB Browser for SQLite" ou a
    janela Database do PyCharm (recurso da assinatura Pro; a JetBrains oferece
    licença educacional gratuita para alunos e professores).
CONCEITOS
    ORM (Object-Relational Mapping): cada TABELA é uma CLASSE e cada LINHA é um OBJETO.
    Chave primária (PK), chave estrangeira (FK), restrição UNIQUE, sessão, commit e rollback.
"""
from datetime import datetime  # data e hora
from pathlib import Path
from sqlalchemy import Float, ForeignKey, String, UniqueConstraint, create_engine, select
from sqlalchemy.exc import IntegrityError  # erro de integridade (ex.: valor duplicado)
from sqlalchemy.orm import DeclarativeBase, Mapped, Session, mapped_column, relationship
# O banco é um único arquivo, criado na mesma pasta deste programa.
BANCO = Path(__file__).with_name("tutorial.db")
class Base(DeclarativeBase):
    """Classe-base: todas as tabelas do nosso modelo herdam dela."""
class Municipio(Base):
    """Tabela "municipio": um município monitorado."""
    __tablename__ = "municipio"  # nome da tabela dentro do banco
    # Mapped[int] diz o tipo da coluna em Python; mapped_column define detalhes no banco.
    # primary_key=True: CHAVE PRIMÁRIA — identifica cada linha; o banco numera sozinho (1, 2, 3...).
    id: Mapped[int] = mapped_column(primary_key=True)
    # String(80): texto de até 80 caracteres; unique=True impede dois municípios com o mesmo nome.
    nome: Mapped[str] = mapped_column(String(80), unique=True)
    uf: Mapped[str] = mapped_column(String(2))  # sigla do estado: 2 letras
    # RELACIONAMENTO 1:N — um município tem uma LISTA de observações.
    # back_populates liga este lado ao atributo "municipio" da classe Observacao.
    # O tipo está entre aspas porque a classe Observacao é definida mais abaixo.
    observacoes: Mapped[list["Observacao"]] = relationship(back_populates="municipio")
class Observacao(Base):
    """Tabela "observacao": uma medição de temperatura em um horário."""
    __tablename__ = "observacao"
    # RESTRIÇÃO DE UNICIDADE COMPOSTA: a combinação (municipio_id, horario) não pode
    # se repetir — impede duas observações do mesmo município no mesmo horário.
    __table_args__ = (UniqueConstraint("municipio_id", "horario"),)
    id: Mapped[int] = mapped_column(primary_key=True)
    # CHAVE ESTRANGEIRA: guarda o "id" de um município existente.
    # O banco não aceita um municipio_id que não exista na tabela municipio.
    municipio_id: Mapped[int] = mapped_column(ForeignKey("municipio.id"))
    horario: Mapped[datetime]  # sem mapped_column: o SQLAlchemy deduz a coluna pelo tipo
    # "float | None" = número OU vazio (NULL no banco): permite registrar dado ausente.
    temperatura: Mapped[float | None] = mapped_column(Float)
    # Lado inverso do relacionamento: obs.municipio devolve o objeto Municipio.
    municipio: Mapped[Municipio] = relationship(back_populates="observacoes")
def preparar(caminho: Path = BANCO):
    """Conecta ao arquivo SQLite e cria as tabelas que ainda não existem."""
    # create_engine prepara a conexão; "sqlite:///" + caminho indica um arquivo SQLite.
    # as_posix() usa barras "/" no caminho, que funcionam também no Windows.
    engine = create_engine(f"sqlite:///{caminho.as_posix()}")
```

exemplos/tutorial/m08_banco.py

```
    Base.metadata.create_all(engine)  # CREATE TABLE para cada classe (se ainda não existir)
    return engine
def popular(engine) -> None:
    """Grava Sapiranga com 4 observações — apenas se ainda não estiver gravado."""
    # "with Session(...) as sessao": abre uma sessão e a fecha automaticamente no fim do bloco.
    with Session(engine) as sessao:
        # select(...).where(...) monta um SELECT com filtro; scalar devolve o 1º resultado ou None.
        if sessao.scalar(select(Municipio).where(Municipio.nome == "Sapiranga")):
            return  # já existe: sair sem gravar de novo (programa IDEMPOTENTE)
        sap = Municipio(nome="Sapiranga", uf="RS")  # cria o objeto (ainda não está no banco)
        # List comprehension: cria uma Observacao para cada par (hora, temperatura).
        # datetime(2026, 10, 21, h) = 21/10/2026 às h horas.
        sap.observacoes = [
            Observacao(horario=datetime(2026, 10, 21, h), temperatura=t)
            for h, t in [(9, 19.0), (12, 24.4), (15, 24.5), (18, 20.2)]
        ]
        sessao.add(sap)  # coloca o município (e suas observações) na sessão
        sessao.commit()  # COMMIT: grava de fato no arquivo do banco
def consultar(engine) -> list[tuple[str, datetime, float]]:
    """Devolve (nome, horário, temperatura) das observações com 20 °C ou mais."""
    with Session(engine) as sessao:
        consulta = (
            select(Municipio.nome, Observacao.horario, Observacao.temperatura)  # colunas desejadas
            .join(Observacao)  # JOIN: liga cada município às suas observações (pela FK)
            .where(Observacao.temperatura >= 20)  # WHERE: filtro
            .order_by(Observacao.horario)  # ORDER BY: ordena pelo horário
        )
        # execute roda o SELECT; list(...) transforma o resultado em lista de tuplas.
        return list(sessao.execute(consulta))
def tentar_duplicar(engine) -> str:
    """Tenta gravar outra observação às 09:00 de 21/10 — o banco deve recusar."""
    with Session(engine) as sessao:
        # municipio_id=1 é Sapiranga; o horário 09:00 já existe para ele.
        sessao.add(Observacao(municipio_id=1, horario=datetime(2026, 10, 21, 9), temperatura=99))
        try:
            sessao.commit()
            return "gravou (não deveria!)"
        except IntegrityError:
            # A restrição UNIQUE foi violada. ROLLBACK desfaz o que estava pendente na sessão.
            sessao.rollback()
            return "recusado pela restrição UNIQUE"
if __name__ == "__main__":
    motor = preparar()  # "motor" (engine) é a conexão com o banco
    popular(motor)
    # Cada item da lista é uma tupla (nome, horario, temp), desempacotada no "for".
    for nome, horario, temp in consultar(motor):
        # {horario:%d/%m %H:%M} formata a data como "21/10 12:00".
        print(f"{nome} {horario:%d/%m %H:%M} -> {temp} °C")
    print("Duplicar 21/10 09:00:", tentar_duplicar(motor))
```

8.7 Explicação do código

- `class Municipio(Base)` com `__tablename__` define a tabela `municipio`; `Mapped[int]` e `mapped_column` definem colunas e tipos.
- `ForeignKey("municipio.id")` cria a chave estrangeira; `relationship` permite navegar de `sap.observacoes` para a lista de observações.
- `UniqueConstraint("municipio_id", "horario")` impede duas observações na mesma hora para o mesmo município.
- `Base.metadata.create_all(engine)` cria as tabelas que não existem (o ClimaAlerta faz isso em `src/database/connection.py`).
- `popular` verifica se Sapiranga já existe antes de inserir — por isso é idempotente.

- `consultar` usa `select ... join ... where ... order_by`, o equivalente a um `SELECT` SQL com filtro.
- `tentar_duplicar` mostra a restrição em ação: o banco recusa e fazemos `rollback`.

8.8 Resultado esperado

Saída real

```
Sapiranga 21/10 12:00 -> 24.4 °C
Sapiranga 21/10 15:00 -> 24.5 °C
Sapiranga 21/10 18:00 -> 20.2 °C
Duplicar 21/10 09:00: recusado pela restrição UNIQUE
```

8.9 Verificação

Execute `test_m08_banco` pelo ▶ em `tests/test_tutorial.py` e depois clique com o botão direito em `tests/test_database.py` > **Run 'pytest in test_database.py'** (testes do banco do ClimaAlerta completo, que verificam as mesmas ideias em escala maior). No Terminal:

Terminal do PyCharm — pasta raiz

```
pytest tests/test_tutorial.py -k m08 -v
pytest tests/test_database.py -v
```

8.10 Erros frequentes e soluções

Erro	Causa	Solução
`IntegrityError: UNIQUE constraint failed`	Dado duplicado	É o comportamento esperado; trate com `try/except` e `rollback`
`OperationalError: no such table`	Tabelas não criadas	Chame `create_all` antes de usar
`database is locked`	Arquivo aberto em outro programa	Feche o DB Browser (ou desconecte na janela Database) e tente de novo
Alterou o modelo e a coluna nova não aparece	`create_all` não altera tabelas existentes	Apague `tutorial.db` (botão direito > Delete) e execute de novo

8.11 Exercícios práticos

1. Acrescente a coluna `umidade` em `Observacao`, apague `tutorial.db` e popule com umidades.
2. Cadastre Porto Alegre com duas observações e escreva uma consulta com a média por município (`func.avg`).
3. Abra `tutorial.db` no visualizador e confira as linhas das duas tabelas.

8.12 Critérios de conclusão

- ☐ O banco é criado, a duplicação é recusada e o teste `m08` passa.
- ☐ Você sabe explicar PK, FK e UNIQUE.

8.13 Desafios de extensão

- Compare `m08_banco.py` com `src/database/models.py`: quais restrições a mais o ClimaAlerta usa?
- Pesquise o *upsert* (`INSERT ... ON CONFLICT DO UPDATE`) usado em `src/database/repositories.py`.

Módulo 9 — Implementação das regras de negócio

9.1 Objetivo de aprendizagem

Transformar uma regra de negócio escrita em português em código: um **indicador local de atenção** para rajadas de vento, com limite configurável, tratamento de dados ausentes e classificação "não oficial".

9.2 Pré-requisitos

Módulo 2.

9.3 Conceitos

- **Regra de negócio:** condição + resultado esperado + exceções. Exemplo (RN007): "se a rajada prevista for maior ou igual ao limite, gerar um indicador com o maior valor, o horário e o número de ocorrências; valores ausentes nunca disparam".
- **Requisito × regra:** o requisito diz *o que* o sistema faz ("calcular indicadores"); a regra diz *como decidir*.
- **Alerta oficial × indicador calculado (RN009):** o alerta oficial é publicado por uma autoridade (INMET). O indicador é um cálculo nosso, com limite demonstrativo. Eles nunca se confundem.
- **Limite inclusivo:** `>=` significa que 60 km/h dispara quando o limite é 60.
- **List comprehension** e **lambda**: formas curtas de criar listas filtradas e funções simples.

9.4 Arquivos a criar ou alterar

Arquivo	Ação
`exemplos/tutorial/m09_regras.py`	Criar

9.5 Passo a passo no PyCharm

1. Crie `m09_regras` em `exemplos/tutorial` e digite o código da seção 9.6.
2. Execute com ▶.
3. Use o **Python Console** do PyCharm (View > Tool Windows > Python Console) para experimentar a função interativamente: digite `from m09_regras import indicador_rajada` e depois `indicador_rajada([{"horario": "x", "rajada": 65}])`. (Se o console não encontrar o módulo, confirme que `exemplos/tutorial` está marcada como Sources Root.)

Equivalente no Terminal do PyCharm:

Terminal do PyCharm — pasta raiz

```
python exemplos/tutorial/m09_regras.py
```

9.6 Código

exemplos/tutorial/m09_regras.py

```
"""Módulo 9 — Regra de negócio: indicador local de atenção para rajadas de vento.
O QUE ESTE ARQUIVO FAZ
    Recebe uma previsão hora a hora (lista de dicionários) e verifica se alguma
    rajada de vento atinge o limite configurado. Se atingir, devolve um
    "indicador" com o maior valor encontrado, o horário, o limite usado, quantas
    horas atingiram o limite e a classificação "INDICADOR CALCULADO (não oficial)".
```

exemplos/tutorial/m09_regras.py

```
A REGRA DE NEGÓCIO, EM PORTUGUÊS
    Condição: rajada prevista >= limite (o limite é INCLUSIVO: 60 km/h dispara).
    Resultado: um indicador com o maior valor, seu horário e o nº de ocorrências.
    Exceções: valores ausentes (None) nunca disparam; sem ocorrências, devolve None.
IMPORTANTE
    Este indicador é um CÁLCULO DIDÁTICO. Não é um alerta oficial (alertas oficiais
    são publicados pelo INMET e pela Defesa Civil) e não garante segurança.
COMO EXECUTAR NO PYCHARM
    Botão ▶ ao lado de  if __name__ == "__main__":  ou  Run 'm09_regras'.
    Os testes deste arquivo são escritos no módulo 13 (test_m09_regras.py).
"""
# Limite DEMONSTRATIVO, em km/h. Não é uma recomendação oficial de segurança.
LIMITE_RAJADA_KMH = 60.0
def indicador_rajada(previsao: list[dict], limite: float = LIMITE_RAJADA_KMH) -> dict | None:
    """Devolve o indicador com a maior rajada >= limite, ou None se nenhuma hora atingir o limite.
    Parâmetros:
        previsao: lista de dicionários, cada um com as chaves "horario" e "rajada".
        limite: valor mínimo (km/h) que dispara o indicador. Padrão: LIMITE_RAJADA_KMH.
    Devolve:
        Um dicionário (o indicador) ou None.  "dict | None" significa "um ou outro".
    """
    # LIST COMPREHENSION (lista por compreensão): cria uma lista nova a partir de outra.
    # Lê-se: "para cada p em previsao, mantenha p SE a rajada não for None E for >= limite".
    # p.get("rajada") devolve None quando a chave não existe (p["rajada"] daria erro).
    # A ORDEM das condições importa: primeiro verificamos "is not None"; só depois
    # comparamos com o limite (comparar None >= 60 causaria TypeError).
    validos = [p for p in previsao if p.get("rajada") is not None and p["rajada"] >= limite]
    # Lista vazia é considerada "falsa": nenhuma hora atingiu o limite.
    if not validos:
        return None
    # max(..., key=...) escolhe o item com o MAIOR valor segundo a função "key".
    # lambda p: p["rajada"] é uma função curta, sem nome, que devolve a rajada de p.
    pior = max(validos, key=lambda p: p["rajada"])
    # O indicador reúne todas as informações exigidas pela regra.
    return {
        "condicao": "Rajadas de vento acima do limite",
        "valor": pior["rajada"],  # o valor mais extremo encontrado
        "limite": limite,  # o limite usado (para quem ler saber o critério)
        "horario": pior["horario"],  # quando ocorre o valor mais extremo
        "ocorrencias": len(validos),  # quantas horas atingiram o limite
        "classificacao": "INDICADOR CALCULADO (não oficial)",  # nunca chamar de "alerta"
    }
if __name__ == "__main__":
    # Previsão de exemplo (valores inventados para o exercício).
    previsao = [
        {"horario": "21/10 15h", "rajada": 35.3},  # abaixo do limite
        {"horario": "21/10 18h", "rajada": 62.0},  # atinge o limite
        {"horario": "21/10 21h", "rajada": None},  # dado ausente: deve ser ignorado
        {"horario": "22/10 00h", "rajada": 71.4},  # atinge o limite e é o maior valor
    ]
    resultado = indicador_rajada(previsao)
    # Se resultado for um dicionário, imprime; se for None, imprime a mensagem.
    print(resultado if resultado else "Nenhum limite atingido.")
```

9.7 Explicação do código

- A *list comprehension* `[p for p in previsao if ...]` mantém apenas as horas com rajada **não nula** e maior ou igual ao limite.
- `max(validos, key=lambda p: p["rajada"])` escolhe a hora com a maior rajada.
- O dicionário devolvido tem todos os campos exigidos para um indicador: condição, valor, limite, horário, ocorrências e classificação.

- O ClimaAlerta completo (`src/services/indicators.py`) generaliza essa ideia para sete regras (IND01–IND07) com limites lidos do `.env`:

Regra	Indicador	Variável	Condição (padrão)	Escopo
IND01	Probabilidade elevada de precipitação	`precipitation_probability`	>= 70 %	próximas 24 h (um indicador com o valor extremo)
IND02	Precipitação horária acima do limite	`precipitation`	>= 10 mm	próximas 24 h (um indicador com o valor extremo)
IND03	Precipitação diária acima do limite	`precipitation_sum`	>= 30 mm	cada dia da previsão
IND04	Rajadas de vento acima do limite	`wind_gusts`	>= 60 km/h	próximas 24 h (um indicador com o valor extremo)
IND05	Temperatura máxima elevada	`temperature_max`	>= 35 °C	cada dia da previsão
IND06	Temperatura mínima baixa	`temperature_min`	<= 3 °C	cada dia da previsão
IND07	Umidade relativa baixa	`humidity`	<= 30 %	próximas 24 h (um indicador com o valor extremo)

9.8 Resultado esperado

Saída real

```
{'condicao': 'Rajadas de vento acima do limite', 'valor': 71.4, 'limite': 60.0, 'horario': '22/10 00h', 'ocorrencias': 2, 'classi
  ↪ ficacao': 'INDICADOR CALCULADO (não oficial)'}
```

9.9 Verificação

Os testes deste módulo serão escritos por você no módulo 13. Por enquanto, confira manualmente: mude a rajada de 22/10 00h para 50 e execute de novo — o valor deve passar a 62,0 e as ocorrências a 1.

9.10 Erros frequentes e soluções

Erro	Causa	Solução
`TypeError: '>=' not supported between 'NoneType' and 'float'`	Comparou valor ausente	Teste `is not None` **antes** de comparar
`ValueError: max() arg is an empty sequence`	Nenhuma hora atingiu o limite	Verifique `if not validos: return None`
Indicador apresentado como "alerta"	Texto incorreto na interface	Sempre exiba a classificação "não oficial"

9.11 Exercícios práticos

1. Escreva `indicador_chuva(previsao, limite=10)` para chuva horária ≥ 10 mm.
2. Altere o limite padrão para 50 km/h. Quantas ocorrências passam a existir?
3. Escreva, em português, a regra de negócio de um indicador de "umidade baixa" com condição, resultado e exceções.

9.12 Critérios de conclusão

- ☐ O indicador é calculado corretamente para os dados de exemplo.
- ☐ Você sabe explicar por que um indicador não é um alerta oficial.

9.13 Desafios de extensão

- Leia o limite de uma variável de ambiente com `os.getenv("LIMITE_RAJADA_KMH", "60")`. No PyCharm, defina a variável em Run > Edit Configurations > m09_regras > **Environment variables**.
- Escreva uma função genérica `indicador(previsao, variavel, operador, limite)` usando o módulo `operator` (sem `eval`).

Módulo 10 — Desenvolvimento da interface

10.1 Objetivo de aprendizagem

Construir uma interface Streamlit que **lê dados do banco**, oferece filtros e exibe métricas, gráfico Plotly e tabela; executá-la pelo PyCharm e conhecer as páginas da interface do ClimaAlerta.

10.2 Pré-requisitos

Módulos 3 e 8 (o arquivo `exemplos/tutorial/tutorial.db` precisa existir).

10.3 Conceitos

- **Separação de responsabilidades:** a interface não deve buscar dados na internet; ela lê o banco. Assim continua funcionando sem internet.
- **Filtros:** widgets (`selectbox`, `slider`) cujos valores filtram o DataFrame.
- **Plotly Express:** gráficos interativos com zoom e dicas (`px.line`).
- **Navegação em várias páginas:** o ClimaAlerta usa `st.navigation` com sete páginas (`app.py`).
- **Sources Root:** marcação do PyCharm que faz o editor reconhecer importações entre arquivos da mesma pasta (`from m08_banco import ...`).

10.4 Arquivos a criar ou alterar

Arquivo	Ação
`exemplos/tutorial/m10_interface.py`	Criar
Run Configurations "m10 - interface" e "ClimaAlerta"	Criar no PyCharm

10.5 Passo a passo no PyCharm

1. Execute `m08_banco.py` (▶) para garantir que o banco existe.
2. Crie `m10_interface` em `exemplos/tutorial` e digite o código da seção 10.6. Se `m08_banco` ficar sublinhado em vermelho, confirme que `exemplos/tutorial` está marcada como **Sources Root** (módulo 1, passo 7).
3. Crie a Run Configuration `m10 - interface`: copie a configuração `m03 - dashboard` (Edit Configurations > Copy Configuration) e troque o parâmetro para `run exemplos/tutorial/m10_interface.py`. Execute.
4. Crie mais uma, `ClimaAlerta`, com o parâmetro `run app.py`, para a interface completa do projeto. Execute-a e navegue pelas 7 páginas.

Equivalente no Terminal do PyCharm:

Terminal do PyCharm — pasta raiz

```
python exemplos/tutorial/m08_banco.py
streamlit run exemplos/tutorial/m10_interface.py
streamlit run app.py
```

10.6 Código

exemplos/tutorial/m10_interface.py

```
"""Módulo 10 — Interface Streamlit que lê o banco de dados criado no módulo 8.
O QUE ESTE ARQUIVO FAZ
    Lê as observações gravadas em tutorial.db, oferece dois filtros (município e
    temperatura mínima) e exibe um cartão com a quantidade de registros, um
    gráfico interativo (Plotly) e a tabela filtrada.
    Princípio importante: a interface LÊ O BANCO; ela não busca dados na internet.
ANTES DE EXECUTAR
    Rode o módulo 8 (m08_banco.py) para criar e popular o banco.
COMO EXECUTAR NO PYCHARM
    a) Terminal (Alt+F12), na pasta raiz:
           streamlit run exemplos/tutorial/m10_interface.py
    b) Ou crie uma Run Configuration (como no módulo 3):
           Module: streamlit
           Script parameters: run exemplos/tutorial/m10_interface.py
           Working directory: pasta raiz do projeto
SOBRE O "from m08_banco import ..."
    O Streamlit coloca a pasta deste arquivo no caminho de importação, então o
    import funciona na execução. Para o EDITOR do PyCharm também reconhecer o
    módulo (sem sublinhado vermelho), clique com o botão direito na pasta
    exemplos/tutorial > Mark Directory as > Sources Root.
"""
import pandas as pd
import plotly.express as px  # gráficos interativos (zoom, dica ao passar o mouse)
import streamlit as st
from m08_banco import Municipio, Observacao, preparar  # reaproveitamos o modelo do módulo 8
from sqlalchemy import select
from sqlalchemy.orm import Session
st.title("□ Observações do banco (tutorial)")
# preparar() conecta ao tutorial.db e cria as tabelas, caso ainda não existam.
motor = preparar()
# ---------------------------------------------------------------------------
# LEITURA DO BANCO
# Monta um SELECT com JOIN (município + observações) e converte o resultado em
# DataFrame. .all() devolve todas as linhas como uma lista de tuplas.
# ---------------------------------------------------------------------------
with Session(motor) as sessao:
    consulta = select(Municipio.nome, Observacao.horario, Observacao.temperatura).join(Observacao)
    df = pd.DataFrame(sessao.execute(consulta).all(), columns=["municipio", "horario", "temperatura"])
motor.dispose()  # fecha as conexões com o banco (os dados já estão no DataFrame)
# Se o banco estiver vazio, orienta o usuário e interrompe a página (st.stop()).
if df.empty:
    st.info("O banco está vazio. Execute primeiro: python exemplos/tutorial/m08_banco.py")
    st.stop()
# ---------------------------------------------------------------------------
# FILTROS
# selectbox devolve o município escolhido; slider devolve um número entre 0 e 40
# (valor inicial 0). sorted(...unique()) lista os municípios sem repetição, em ordem.
# ---------------------------------------------------------------------------
municipio = st.selectbox("Município", sorted(df["municipio"].unique()))
minima = st.slider("Mostrar temperaturas a partir de (°C)", 0, 40, 0)
# Duas condições combinadas com "&" (E lógico). Cada condição fica entre parênteses.
filtrado = df[(df["municipio"] == municipio) & (df["temperatura"] >= minima)]
# ---------------------------------------------------------------------------
# EXIBIÇÃO
# ---------------------------------------------------------------------------
st.metric("Registros exibidos", len(filtrado))  # cartão com a contagem
# px.line cria o gráfico de linha; markers=True mostra um ponto em cada observação.
grafico = px.line(filtrado, x="horario", y="temperatura", markers=True, title=f"Temperatura — {municipio}")
st.plotly_chart(grafico)
st.dataframe(filtrado, hide_index=True)  # tabela com as linhas filtradas
```

10.7 Explicação do código

- `from m08_banco import ...` reaproveita o modelo e a função `preparar` do módulo 8 (o Streamlit coloca a pasta do script no caminho de importação).
- A consulta `select(...).join(Observacao)` lê o banco; o resultado vira um DataFrame. `motor.dispose()` fecha as conexões.
- `st.stop()` interrompe a página quando o banco está vazio, com uma orientação ao usuário.
- O filtro combina duas condições com `&` (e lógico).
- `px.line(..., markers=True)` cria o gráfico interativo; `st.plotly_chart` o exibe.

10.8 Resultado esperado

Uma página com o seletor de município (Sapiranga), um controle deslizante de temperatura mínima, o cartão "Registros exibidos: 4", o gráfico e a tabela. Com o controle em 22 °C, restam 2 registros.

A interface completa do ClimaAlerta, com dados reais:



Figura 3 — Página Previsão do ClimaAlerta (gráficos Plotly e tabela)

10.9 Verificação

Execute `test_m10_interface_le_o_banco` pelo ▶ em `tests/test_tutorial.py` e depois `tests/test_ui_pages.py` (botão direito > Run). No Terminal:

Terminal do PyCharm — pasta raiz

```
pytest tests/test_tutorial.py -k m10 -v
pytest tests/test_ui_pages.py -v
```

10.10 Erros frequentes e soluções

Erro	Causa	Solução
`ModuleNotFoundError: No module named 'm08_banco'`	Arquivo do módulo 8 ausente ou com outro nome	Confira `exemplos/tutorial/m08_banco.py`
`m08_banco` sublinhado, mas executa	Pasta não marcada como Sources Root	Botão direito na pasta > Mark Directory as > Sources Root
Página mostra "O banco está vazio"	`m08_banco.py` não executado	Execute-o antes
Gráfico vazio após filtrar	Filtro muito restritivo	Reduza o valor do controle deslizante

10.11 Exercícios práticos

1. Acrescente um `st.metric` com a temperatura máxima do filtro.
2. Troque o gráfico por `px.bar` e compare a leitura.
3. No ClimaAlerta, navegue por todas as páginas e anote: qual página mostra a última coleta de cada fonte?

10.12 Critérios de conclusão

- ☐ Sua interface lê o banco, filtra e o teste `m10` passa.
- ☐ As configurações `m10 - interface` e `ClimaAlerta` funcionam.
- ☐ Você navegou pelas 7 páginas do ClimaAlerta.

10.13 Desafios de extensão

- Acrescente um botão `st.download_button` para exportar o DataFrame filtrado em CSV.
- Separe sua interface em duas páginas usando `st.navigation` e `st.Page`, como em `app.py`.

Módulo 11 — Integração dos módulos

11.1 Objetivo de aprendizagem

Executar o ClimaAlerta completo com dados reais a partir do PyCharm, entender como as camadas se conectam (interface → serviços → integrações → banco) e relacionar cada parte com o que você construiu nos módulos 2 a 10.

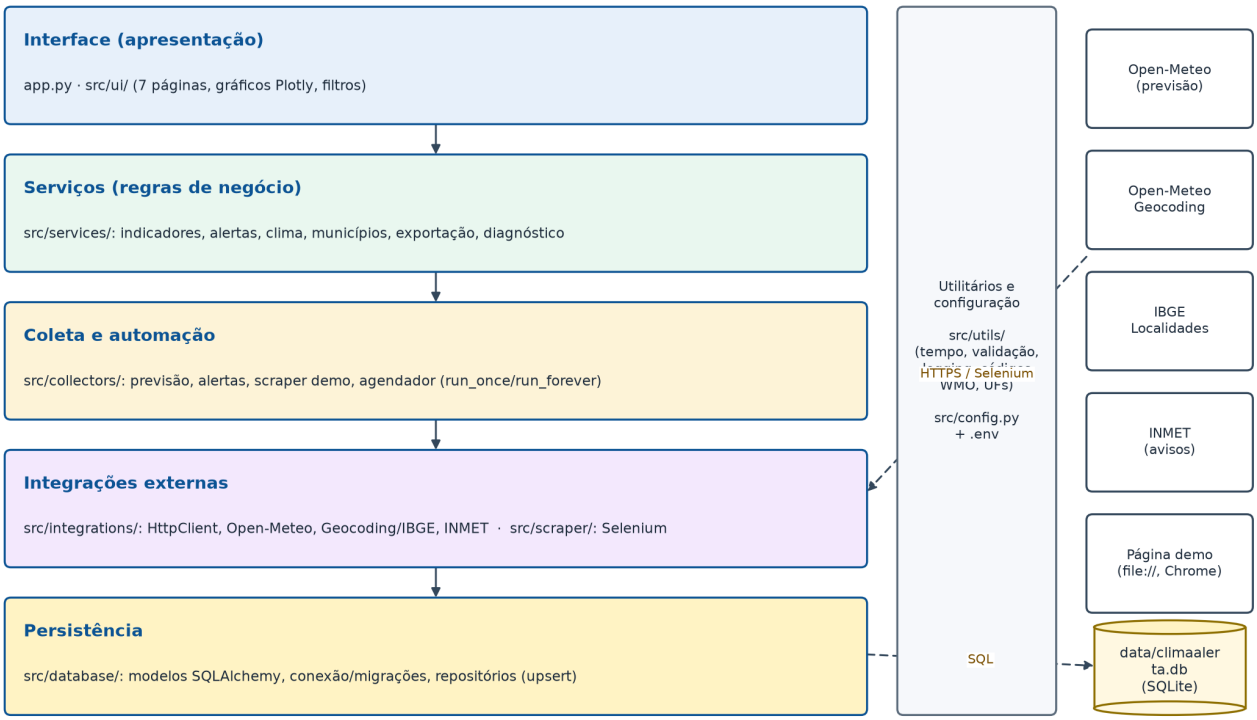
11.2 Pré-requisitos

Módulos 1 a 10. Internet.

11.3 Conceitos

- **Arquitetura em camadas:** cada pasta de `src/` tem uma responsabilidade. A interface lê o banco por meio dos serviços; só os coletores chamam as APIs.
- **Configuração externa:** `src/config.py` lê o `.env`. Mudar um limite não exige alterar o código.
- **Injeção de dependências:** `src/services/context.py` cria os clientes HTTP e das APIs em um só lugar; nos testes, eles são trocados por dublês.
- **Upsert:** gravar sem duplicar — se o registro existe, é atualizado.
- **Dados oficiais × calculados:** avisos do INMET vão para `weather_alert`; indicadores são calculados na hora e exibidos em outra seção.
- **Navegação no código com o PyCharm:** Ctrl+B (ir para a definição), Alt+F7 (onde é usado), Ctrl+F12 (estrutura do arquivo), Ctrl+Shift+N (abrir arquivo pelo nome).

Arquitetura em camadas do ClimaAlerta



Regra de dependência: cada camada usa apenas as camadas abaixo dela. A interface nunca chama APIs diretamente: lê o banco por meio dos serviços.

Figura 4 — Camadas do ClimaAlerta

11.4 Arquivos a criar ou alterar

Arquivo	Ação
`data/climaalerta.db`	Criado pela inicialização
`.env`	Alterar um limite (exercício)
Run Configurations dos scripts	Criadas pelo PyCharm ao executar

11.5 Passo a passo no PyCharm

1. Abra `scripts/inicializar_banco.py` e clique em ▶ (botão direito no editor > Run). O banco é criado e Sapiranga é cadastrado.
2. Abra `scripts/coletar_dados.py` e clique em ▶. A janela Run mostra o resultado de cada fonte.
3. Escolha a configuração `ClimaAlerta` (criada no módulo 10) no topo da janela e clique em ▶. Navegue pelas páginas.
4. **Explore a arquitetura:** abra `app.py`, posicione o cursor em `get_services` e pressione Ctrl+B para ir à definição; dentro dela, use Ctrl+B em `build_services`. Em `src/collectors/weather_collector.py`, pressione Ctrl+F12 para ver a estrutura do arquivo. Clique com o botão direito em `collect_weather` > **Find Usages** (Alt+F7) para ver quem chama essa função.
5. **Repositório-esqueleto:** se a turma estiver usando o esqueleto distribuído pelo professor, abra **View > Tool Windows > TODO** (Alt+6): a janela lista todas as funções com `TODO: implementar` — é o seu mapa de tarefas.

Equivalente no Terminal do PyCharm:

Terminal do PyCharm — pasta raiz

```
python scripts/inicializar_banco.py
python scripts/coletar_dados.py
streamlit run app.py
```

11.6 Código

O ponto de entrada da interface registra as páginas:

app.py (ClimaAlerta completo)

```
"""ClimaAlerta — ponto de entrada da interface.
Execução (na raiz do projeto, com o ambiente virtual ativo):
    streamlit run app.py
"""
from __future__ import annotations
import streamlit as st
from src.ui import (
    page_alertas,
    page_aprenda,
    page_diagnostico,
    page_historico,
    page_municipios,
    page_painel,
    page_previsao,
)
from src.ui.common import get_services
st.set_page_config(page_title="ClimaAlerta", page_icon="□□", layout="wide")
get_services()  # inicializa banco, logging e município de demonstração
pages = [
    st.Page(page_painel.render, title="Painel", icon="□", url_path="painel", default=True),
    st.Page(page_previsao.render, title="Previsão", icon="□", url_path="previsao"),
    st.Page(page_alertas.render, title="Central de alertas", icon="□", url_path="alertas"),
    st.Page(page_municipios.render, title="Municípios", icon="□", url_path="municipios"),
```

app.py (ClimaAlerta completo)

```
    st.Page(page_historico.render, title="Histórico", icon="□□", url_path="historico"),
    st.Page(page_diagnostico.render, title="Diagnóstico", icon="□", url_path="diagnostico"),
    st.Page(page_aprenda.render, title="Aprenda", icon="□", url_path="aprenda"),
]
st.sidebar.markdown("## □□ ClimaAlerta")
st.sidebar.caption("Monitoramento meteorológico educacional")
st.navigation(pages).run()
```

O coletor de previsões integra API, validação, banco e registro de coleta:

src/collectors/weather_collector.py

```
"""Coletor de previsões e condições atuais (Open-Meteo -> SQLite).
Cada município é coletado em sua própria transação. Se a fonte falhar, a
transação de dados não é aberta e os registros válidos já existentes
permanecem intactos (RN010). Toda tentativa gera um ``CollectionLog``.
"""
from __future__ import annotations
import logging
from dataclasses import dataclass
from src.database import repositories as repo
from src.database.connection import session_scope
from src.database.models import CollectionLog
from src.integrations.http_client import InvalidResponseError, SourceUnavailableError
from src.integrations.open_meteo import SOURCE_NAME, OpenMeteoClient
logger = logging.getLogger(__name__)
@dataclass
class CollectionResult:
    source: str
    municipality: str | None
    status: str
    records: int
    error: str | None = None
def finish_log(database_url: str, log_id: int, status: str, records: int, error: str | None) -> None:
    with session_scope(database_url) as session:
        log = session.get(CollectionLog, log_id)
        repo.finish_collection_log(log, status, records, error)
def collect_weather(
    database_url: str,
    client: OpenMeteoClient,
    municipality_ids: list[int] | None = None,
    forecast_days: int = 7,
) -> list[CollectionResult]:
    with session_scope(database_url) as session:
        targets = [
            (m.id, m.label, m.latitude, m.longitude, m.timezone)
            for m in repo.list_municipalities(session, active_only=municipality_ids is None)
            if municipality_ids is None or m.id in municipality_ids
        ]
    results: list[CollectionResult] = []
    for mid, label, lat, lon, tz in targets:
        with session_scope(database_url) as session:
            log_id = repo.start_collection_log(session, SOURCE_NAME, "PREVISAO", mid).id
        try:
            bundle = client.fetch_forecast(lat, lon, forecast_days, tz)
            with session_scope(database_url) as session:
                records = 0
                if bundle.current:
                    records += repo.upsert_observation(session, mid, bundle.current)
                records += repo.upsert_hourly_forecasts(session, mid, bundle.hourly)
                records += repo.upsert_daily_forecasts(session, mid, bundle.daily)
                repo.finish_collection_log(session.get(CollectionLog, log_id), "SUCESSO", records)
            logger.info("Coleta Open-Meteo concluída para %s: %d registros", label, records)
            results.append(CollectionResult(SOURCE_NAME, label, "SUCESSO", records))
        except (SourceUnavailableError, InvalidResponseError) as exc:
            logger.error("Falha na coleta Open-Meteo para %s: %s", label, exc)
```

src/collectors/weather_collector.py

```
            finish_log(database_url, log_id, "FALHA", 0, str(exc))
            results.append(CollectionResult(SOURCE_NAME, label, "FALHA", 0, str(exc)))
    return results
```

11.7 Explicação do código

- `app.py` chama `get_services()` (configuração, logging, banco e Sapiranga) e registra as páginas com `st.Page`.
- `collect_weather` lê os municípios ativos, cria um `CollectionLog` com situação inicial "FALHA", chama a API e, **somente se tudo der certo**, grava observação, previsão horária e diária em uma transação e marca "SUCESSO".
- Se a API falhar, o `except` registra a falha e os dados antigos continuam no banco — por isso a interface funciona sem internet.
- Compare: `fetch_forecast` corresponde ao seu `buscar_condicoes` (módulo 6); `upsert_*` ao seu `popular` (módulo 8); a página Painel à sua interface (módulo 10); `evaluate_indicators` ao seu `indicador_rajada` (módulo 9).

11.8 Resultado esperado

Saída real de scripts/coletar_dados.py (21/10/2026)

```
INMET: SUCESSO, 10 registros
Open-Meteo (Sapiranga - RS): SUCESSO, 176 registros
```

176 = 1 observação atual + 168 horas + 7 dias. No navegador, o Painel mostra:

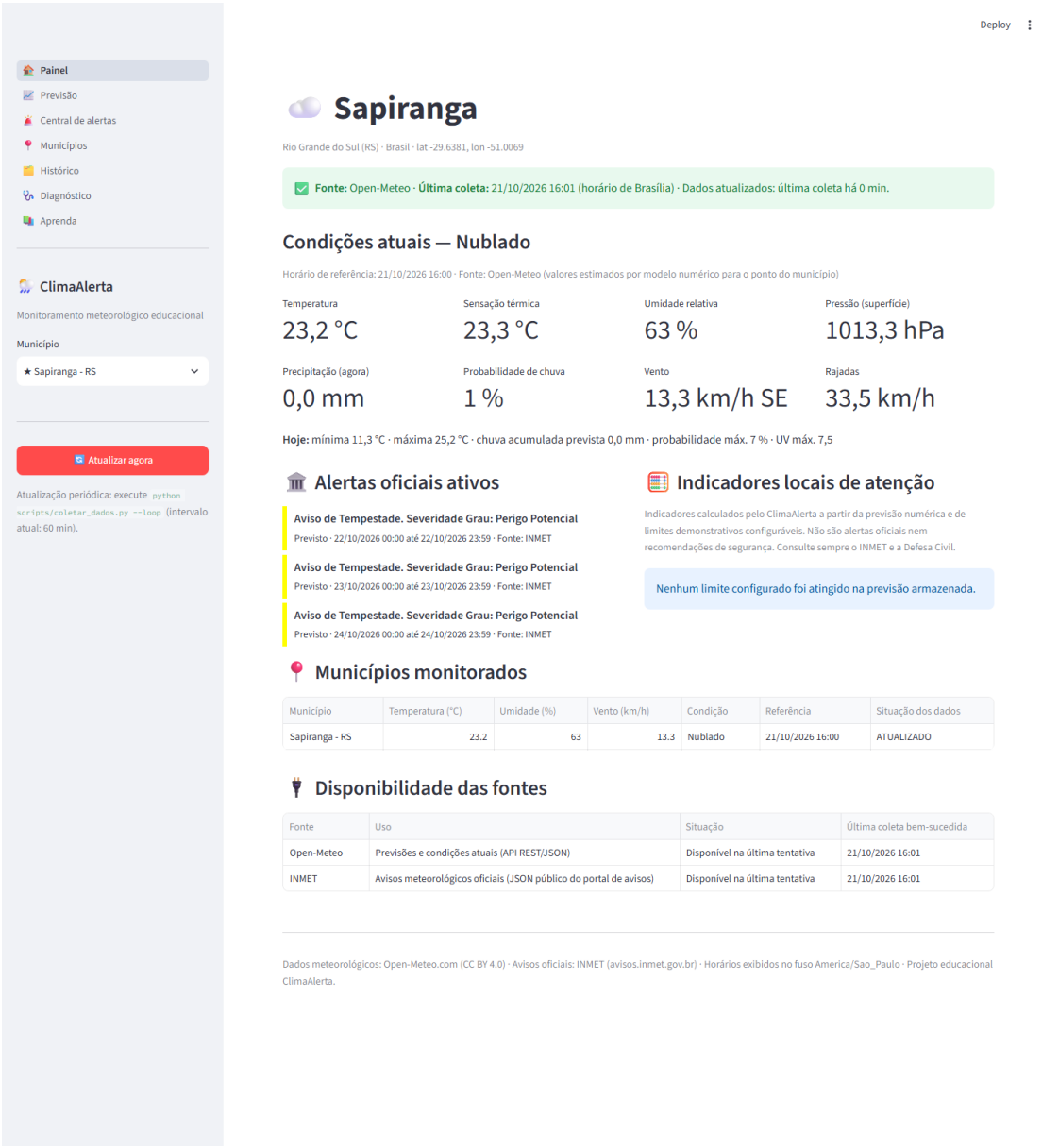


Figura 5 — Painel do ClimaAlerta com dados reais

11.9 Verificação

Clique com o botão direito em `tests/test_collectors.py` > **Run 'pytest in test_collectors.py'** e repita com `tests/test_ui_pages.py`. Na interface, abra **Diagnóstico** e confira que as duas fontes estão "Disponível na última tentativa" e que `weather_forecast` tem 7 registros. No Terminal:

Terminal do PyCharm — pasta raiz

```
pytest tests/test_collectors.py tests/test_ui_pages.py -v
```

11.10 Erros frequentes e soluções

Erro	Causa	Solução
Fonte com "FALHA" no Diagnóstico	Sem internet ou fonte fora do ar	Veja a mensagem; tente mais tarde (os dados antigos continuam)
Painel sem dados	Nenhuma coleta realizada	Clique em "Atualizar agora" ou execute `coletar_dados.py`
Limite alterado no `.env` sem efeito	Configuração lida na inicialização	Pare (■) e execute a configuração `ClimaAlerta` de novo
`ModuleNotFoundError: No module named 'src'` ao executar um script	Working directory diferente da raiz	Defina a pasta raiz em Run > Edit Configurations > Working directory

11.11 Exercícios práticos

1. No `.env`, altere `LIMITE_PROBABILIDADE_CHUVA=70` para `5`, reinicie a interface e observe a seção de indicadores. Depois volte para 70.
2. Cadastre outro município na página Municípios, clique em "Atualizar agora" e compare os dois no Painel.
3. Na Central de alertas, filtre por severidade "Perigo Potencial". Qual o link oficial do primeiro aviso?
4. Use Alt+F7 em `evaluate_indicators` e liste os arquivos que a utilizam.

11.12 Critérios de conclusão

- ☐ Coleta executada com SUCESSO nas duas fontes.
- ☐ Você navegou pelo código com Ctrl+B e Alt+F7.
- ☐ Você explica o caminho de um dado da API até a tela.

11.13 Desafios de extensão

- Leia `src/services/alert_service.py` e explique como o filtro por município usa o código IBGE.
- Desenhe o caminho do botão "Atualizar agora" e compare com o diagrama de sequência do documento 03.

Módulo 12 — Atualização automatizada

12.1 Objetivo de aprendizagem

Automatizar a coleta com um laço temporizado, tratar falhas sem interromper o processo, passar argumentos a um programa pelo PyCharm e usar o agendador do ClimaAlerta.

12.2 Pré-requisitos

Módulos 6 e 11.

12.3 Conceitos

- **Laço temporizado:** repetir uma tarefa e esperar (`time.sleep`) entre as repetições.
- **Robustez:** um erro em uma rodada (por exemplo, sem internet) deve ser registrado, e o laço deve continuar.
- **Cortesia com a fonte:** não consultar a API com frequência exagerada. O ClimaAlerta impõe intervalo mínimo de 15 minutos.
- **Argumentos de linha de comando:** `argparse` permite `--vezes 3 --intervalo 20`. No PyCharm, eles são informados no campo **Script parameters** da Run Configuration.
- **Agendadores do sistema:** Agendador de Tarefas (Windows) e `cron` (Linux) executam um script em horários definidos, mesmo com o PyCharm fechado.

12.4 Arquivos a criar ou alterar

Arquivo	Ação
`exemplos/tutorial/m12_agendador.py`	Criar (usa o `m06_api_open_meteo.py`)
Run Configurations `m12_agendador` e `coletar_dados --loop`	Ajustar no PyCharm

12.5 Passo a passo no PyCharm

1. Crie `m12_agendador` em `exemplos/tutorial` e digite o código da seção 12.6.
2. Execute uma vez com ▶ (usará os valores padrão: 3 vezes, 20 s).
3. Abra **Run > Edit Configurations...**, selecione `m12_agendador` e preencha **Script parameters** com `--vezes 3 --intervalo 5`. Clique em OK e execute de novo (Shift+F10).
4. Para a coleta periódica do sistema: abra `scripts/coletar_dados.py`, execute uma vez com ▶ e, em Edit Configurations, preencha **Script parameters** com `--loop --intervalo 15` e renomeie a configuração para `coletar_dados --loop`. Execute: o processo fica rodando e coleta a cada 15 minutos. Para parar, clique em ■ (Ctrl+F2).

Equivalente no Terminal do PyCharm:

Terminal do PyCharm — pasta raiz

```
python exemplos/tutorial/m12_agendador.py --vezes 3 --intervalo 20
python scripts/coletar_dados.py --loop --intervalo 15
```

Para agendar no sistema operacional, veja o documento 09 (Implantação e operação).

12.6 Código

exemplos/tutorial/m12_agendador.py

```
"""Módulo 12 — Atualização automatizada com um laço temporizado.
O QUE ESTE ARQUIVO FAZ
    Consulta a API do módulo 6 várias vezes, esperando um intervalo entre as
    consultas. Se uma rodada falhar (por exemplo, sem internet), registra a falha
    e CONTINUA nas próximas rodadas, em vez de encerrar o programa.
COMO EXECUTAR NO PYCHARM
    Este programa recebe ARGUMENTOS (--vezes e --intervalo). Opções:
    a) Terminal (Alt+F12), na pasta raiz:
           python exemplos/tutorial/m12_agendador.py --vezes 3 --intervalo 20
    b) Run Configuration: execute uma vez com ▶; depois, em
       Run > Edit Configurations... > m12_agendador, preencha
       Script parameters:  --vezes 3 --intervalo 20
    Para interromper: quadrado vermelho ■ na janela Run (ou Ctrl+C no Terminal).
"""
import argparse  # lê argumentos da linha de comando (--vezes 3 ...)
import time  # time.sleep pausa o programa
from datetime import datetime  # para mostrar o horário de cada rodada
from m06_api_open_meteo import buscar_condicoes  # reaproveita a função do módulo 6
def executar(vezes: int, intervalo: float, coletar=buscar_condicoes, esperar=time.sleep) -> list[dict]:
    """Executa "vezes" rodadas de coleta, esperando "intervalo" segundos entre elas.
    Parâmetros:
        vezes: quantas rodadas executar.
        intervalo: segundos de espera entre uma rodada e a próxima.
        coletar: função que busca os dados (padrão: a API real do módulo 6).
        esperar: função que espera (padrão: time.sleep).
            Os dois últimos parâmetros permitem que o TESTE troque a internet e a
            espera por funções falsas — o teste roda em milissegundos.
    Devolve:
        A lista com os dados das rodadas que deram certo.
    """
    resultados = []
    # range(1, vezes + 1) gera 1, 2, ..., vezes (o último número não é incluído, por isso +1).
    for rodada in range(1, vezes + 1):
        try:
            dados = coletar(-29.63806, -51.00694)  # coordenadas de Sapiranga - RS
            resultados.append(dados)
            # {datetime.now():%H:%M:%S} mostra a hora atual, por exemplo 15:45:44.
            print(f"[{datetime.now():%H:%M:%S}] rodada {rodada}: {dados['temperatura']} °C")
        except Exception as erro:
            # "Exception" captura QUALQUER erro desta rodada. Mostramos o problema e
            # seguimos: uma falha isolada não pode derrubar a coleta automática.
            print(f"[{datetime.now():%H:%M:%S}] rodada {rodada}: falha ({erro})")
        # Espera entre as rodadas — mas não depois da última.
        if rodada < vezes:
            esperar(intervalo)
    return resultados
if __name__ == "__main__":
    # argparse define quais argumentos o programa aceita, com valores padrão e ajuda.
    parser = argparse.ArgumentParser(description="Coleta repetida (tutorial)")
    parser.add_argument("--vezes", type=int, default=3)
    parser.add_argument("--intervalo", type=float, default=20, help="segundos entre as coletas")
    args = parser.parse_args()  # lê o que foi digitado (ou os padrões)
    try:
        # max(5.0, ...) garante pelo menos 5 segundos: cortesia com o serviço gratuito da API.
        executar(args.vezes, max(5.0, args.intervalo))
    except KeyboardInterrupt:
        # Ctrl+C (ou o botão ■ do PyCharm) gera KeyboardInterrupt: encerramos com uma mensagem.
        print("Interrompido pelo usuário.")
```

Agendador do ClimaAlerta completo:

src/collectors/scheduler.py

```
"""Atualização manual e periódica.
* ``run_once``: executa uma rodada completa (avisos + previsões).
```

src/collectors/scheduler.py

```
* ``run_forever``: repete a rodada a cada N minutos até ser interrompido
  (Ctrl+C). ``max_cycles`` existe para os testes.
"""
from __future__ import annotations
import logging
import time
from collections.abc import Callable
from src.collectors.alerts_collector import collect_alerts
from src.collectors.weather_collector import CollectionResult, collect_weather
from src.database.connection import session_scope
from src.services.context import AppServices
from src.services.municipality_service import ensure_default_municipality
logger = logging.getLogger(__name__)
def run_once(services: AppServices, municipality_ids: list[int] | None = None) -> list[CollectionResult]:
    url = services.database_url
    with session_scope(url) as session:
        ensure_default_municipality(session)
    results = [collect_alerts(url, services.inmet)]
    results += collect_weather(url, services.open_meteo, municipality_ids, services.settings.forecast_days)
    ok = sum(r.status == "SUCESSO" for r in results)
    logger.info("Rodada de coleta finalizada: %d/%d com sucesso", ok, len(results))
    return results
def run_forever(
    services: AppServices,
    interval_minutes: int | None = None,
    max_cycles: int | None = None,
    sleep: Callable[[float], None] = time.sleep,
) -> int:
    interval = (interval_minutes or services.settings.update_interval_minutes) * 60
    cycles = 0
    logger.info("Atualização periódica iniciada (intervalo de %d min). Ctrl+C para parar.", interval // 60)
    try:
        while max_cycles is None or cycles < max_cycles:
            try:
                run_once(services)
            except Exception:  # um erro inesperado não pode derrubar o agendador
                logger.exception("Erro inesperado na rodada de coleta")
            cycles += 1
            if max_cycles is not None and cycles >= max_cycles:
                break
            sleep(interval)
    except KeyboardInterrupt:
        logger.info("Atualização periódica interrompida pelo usuário.")
    return cycles
```

12.7 Explicação do código

- `executar` recebe as funções `coletar` e `esperar` como parâmetros. No uso normal são a API e `time.sleep`; no teste, funções falsas — o teste roda em milissegundos.
- O `try/except` dentro do laço registra a falha e segue para a próxima rodada.
- `max(5.0, args.intervalo)` impede intervalos menores que 5 s no tutorial.
- No ClimaAlerta, `run_forever` faz o mesmo com `run_once`, usa o intervalo do `.env` (mínimo de 15 min) e trata `KeyboardInterrupt` (Ctrl+C ou o botão ■).

12.8 Resultado esperado

Saída real (intervalo de 5 s)

```
[15:45:44] rodada 1: 23.6 °C
[15:45:50] rodada 2: 23.6 °C
```

Sem internet, a rodada mostra "falha (...)" e o programa continua.

12.9 Verificação

Execute `test_m12_agendador_continua_apos_falha` e `test_arquivos_do_tutorial_existem` pelo ▶ em `tests/test_tutorial.py`. A partir deste módulo, o teste de arquivos também deve passar. No Terminal:

Terminal do PyCharm — pasta raiz

```
pytest tests/test_tutorial.py -k "m12 or arquivos" -v
pytest tests/test_collectors.py -k run_forever -v
```

12.10 Erros frequentes e soluções

Erro	Causa	Solução
`ModuleNotFoundError: m06_api_open_meteo`	Arquivo do módulo 6 ausente	Confira `exemplos/tutorial/m06_api_open_meteo.py`
"error: unrecognized arguments"	Parâmetros digitados errado	Confira o campo Script parameters (`--vezes 3 --intervalo 5`)
Programa "parado"	Está esperando o intervalo	É normal; aguarde ou clique em ■
Coleta para quando o computador hiberna	Processo suspenso	Use o agendador do sistema ou desative a hibernação no laboratório

12.11 Exercícios práticos

1. Execute com o Wi-Fi desligado em uma das rodadas e observe a mensagem de falha.
2. Acrescente a umidade à linha impressa.
3. Deixe a configuração `coletar_dados --loop` rodando durante uma aula e veja o histórico crescer na página Histórico.

12.12 Critérios de conclusão

- ☐ O laço executa as rodadas e continua após falhas; o teste `m12` passa.
- ☐ Você informou argumentos pelo campo Script parameters.
- ☐ Você sabe interromper o `--loop` com ■.

12.13 Desafios de extensão

- Grave cada rodada em um arquivo CSV com data e hora.
- Crie a tarefa do Agendador do Windows (ou o `cron`) descrita no documento 09.

Módulo 13 — Testes e depuração no PyCharm

13.1 Objetivo de aprendizagem

Escrever testes automatizados com **pytest**, executá-los e analisá-los na interface de testes do PyCharm, executar a suíte completa do ClimaAlerta, ler mensagens de erro (*traceback*) e depurar programas com o **depurador** do PyCharm.

13.2 Pré-requisitos

Módulo 9 (`m09_regras.py`).

13.3 Conceitos

- **Teste automatizado:** código que executa outro código e confere o resultado com `assert`.
- **Tipos:** unitário (uma função), integração (várias partes, como banco + coletor), interface, aceitação (ligado a um caso de uso).
- **Casos importantes:** caminho feliz, limites (exatamente 60 km/h), dados ausentes (`None`), entradas inválidas, falhas de rede.
- **Dublês (*mocks*):** objetos falsos que substituem a internet ou o banco nos testes.
- **Traceback:** relatório de erro; leia de **baixo para cima** — a última linha diz o tipo do erro; as linhas acima mostram onde ocorreu. No PyCharm, os nomes de arquivo do traceback são **links**: clique para ir à linha.
- **Depurador:** pausa o programa em um **ponto de parada** e permite executar linha a linha e inspecionar variáveis.

Ação do depurador	Atalho
Marcar/desmarcar ponto de parada	Clique na margem ou Ctrl+F8
Depurar o arquivo atual	Botão direito > Debug '<arquivo>'
Executar a próxima linha (Step Over)	F8
Entrar na função chamada (Step Into)	F7
Sair da função atual (Step Out)	Shift+F8
Continuar até o próximo ponto de parada (Resume)	F9
Avaliar uma expressão	Alt+F8
Parar	Ctrl+F2

13.4 Arquivos a criar ou alterar

Arquivo	Ação
`exemplos/tutorial/test_m09_regras.py`	Criar

13.5 Passo a passo no PyCharm

1. Crie `test_m09_regras` em `exemplos/tutorial` (New > Python File) e digite o código da seção 13.6.
2. Repare nos ▶ verdes ao lado de cada `def test_...`. Clique no ▶ da primeira linha do arquivo para executar todos os testes do arquivo.
3. A janela Run mostra uma **árvore de testes**: ✓ verde (passou) ou ✗ vermelho (falhou). Clique em um teste para ver sua saída. Os botões da janela permitem **Rerun Failed Tests** (reexecutar só os que falharam) e **Toggle auto-test** (reexecutar sozinho ao editar).
4. Execute a suíte completa do projeto: clique com o botão direito na pasta `tests` > **Run 'pytest in tests'**. Ela leva cerca de 1 minuto.

5. **"Teste do teste":** em `m09_regras.py`, troque `>=` por `>` e execute os testes de novo. O teste `test_limite_e_inclusivo` deve **falhar**; clique nele e leia a mensagem de `assert`. Desfaça a alteração (Ctrl+Z) e execute de novo.
6. **Depurar um teste:** clique na margem da linha `pior = max(...)` em `m09_regras.py` (ponto de parada). Clique com o botão direito no ▶ de `test_dispara_com_o_maior_valor` > **Debug**. Quando o programa pausar, observe a variável `validos` na janela Debug, pressione **Alt+F8** e avalie `len(validos)`. Pressione **F9** para terminar.

Equivalente no Terminal do PyCharm:

Terminal do PyCharm — pasta raiz

```
pytest exemplos/tutorial/test_m09_regras.py -v
pytest -v
pytest -v -x
pytest --cov=src --cov-report=term
ruff check .
```

- `pytest -v` executa **todos** os testes do projeto (inclusive os seus); `-x` para no primeiro erro.
- `--cov` mede quanto do código foi executado pelos testes.
- `ruff check .` procura problemas de estilo e erros comuns. O PyCharm também mostra avisos no editor; **Code > Inspect Code...** analisa o projeto inteiro.

13.6 Código

exemplos/tutorial/test_m09_regras.py

```
"""Módulo 13 — Testes automatizados da regra de negócio do módulo 9.
O QUE ESTE ARQUIVO FAZ
    Cada função que começa com "test_" é um TESTE: executa a função
    indicador_rajada com dados preparados e confere o resultado com "assert".
    Se o "assert" for falso, o teste FALHA e o pytest mostra o motivo.
COMO EXECUTAR NO PYCHARM
    - Clique no ▶ verde ao lado de qualquer "def test_..." para rodar só aquele teste,
      ou no ▶ ao lado da primeira linha para rodar todos os testes do arquivo.
    - Ou clique com o botão direito no arquivo > Run 'pytest in test_m09_regras.py'.
    - O resultado aparece na janela Run: ✓ verde = passou; ✗ vermelho = falhou.
    Se o PyCharm não mostrar a opção pytest, configure: Settings (Ctrl+Alt+S) >
    Tools > Python Integrated Tools > Testing > Default test runner: pytest.
    Terminal (Alt+F12), na pasta raiz:
        pytest exemplos/tutorial/test_m09_regras.py -v
O QUE TESTAR (casos importantes)
    caminho feliz, valor exatamente no limite, valor abaixo do limite,
    dado ausente (None) e a classificação "não oficial".
"""
import sys
from pathlib import Path
# Coloca a pasta deste arquivo no caminho de importação do Python, para que o
# "import m09_regras" funcione qualquer que seja a pasta de onde o pytest é chamado.
sys.path.insert(0, str(Path(__file__).parent))
from m09_regras import indicador_rajada  # noqa: E402  (import depois do sys.path, de propósito)
def test_dispara_com_o_maior_valor():
    """Duas horas acima do limite: o indicador deve trazer a MAIOR rajada e contar 2 ocorrências."""
    previsao = [{"horario": "a", "rajada": 61}, {"horario": "b", "rajada": 80}]
    resultado = indicador_rajada(previsao)
    assert resultado["valor"] == 80  # o maior valor
    assert resultado["ocorrencias"] == 2  # as duas horas contam
def test_limite_e_inclusivo():
    """Exatamente 60 km/h deve disparar, porque a regra usa ">=" (maior OU IGUAL)."""
    assert indicador_rajada([{"horario": "a", "rajada": 60}]) is not None
def test_nao_dispara_abaixo_do_limite():
    """59,9 km/h está abaixo de 60: nenhum indicador (None)."""
    assert indicador_rajada([{"horario": "a", "rajada": 59.9}]) is None
def test_valor_ausente_nunca_dispara():
```

exemplos/tutorial/test_m09_regras.py

```
    """Um dado ausente (None) não pode gerar indicador nem causar erro."""
    assert indicador_rajada([{"horario": "a", "rajada": None}]) is None
def test_classificacao_nao_oficial():
    """Todo indicador precisa se identificar como cálculo NÃO oficial."""
    resultado = indicador_rajada([{"horario": "a", "rajada": 90}])
    assert "não oficial" in resultado["classificacao"]  # "in" verifica se o texto contém o trecho
```

13.7 Explicação do código

- Cada função que começa com `test_` é um teste; o pytest (e o PyCharm) a encontra sozinho.
- `sys.path.insert(...)` permite importar `m09_regras` a partir da pasta do teste, qualquer que seja a pasta de trabalho.
- Os cinco testes cobrem: maior valor e contagem, limite inclusivo, valor abaixo do limite, dado ausente e a classificação "não oficial" — exatamente as condições da regra de negócio do módulo 9.
- As docstrings dos testes aparecem na árvore de testes e explicam o que cada um verifica.

13.8 Resultado esperado

Saída real (Terminal); no PyCharm, 5 testes com ✓ verde

```
exemplos/tutorial/test_m09_regras.py::test_dispara_com_o_maior_valor PASSED [ 20%]
exemplos/tutorial/test_m09_regras.py::test_limite_e_inclusivo PASSED     [ 40%]
exemplos/tutorial/test_m09_regras.py::test_nao_dispara_abaixo_do_limite PASSED [ 60%]
exemplos/tutorial/test_m09_regras.py::test_valor_ausente_nunca_dispara PASSED [ 80%]
exemplos/tutorial/test_m09_regras.py::test_classificacao_nao_oficial PASSED [100%]
============================== 5 passed ==============================
```

Para a suíte completa, a quantidade real de testes aprovados está no documento 08 (Plano e relatório de testes).

13.9 Verificação

1. Todos os testes passam com **Run 'pytest in tests'**.
2. O "teste do teste" (passo 5) fez `test_limite_e_inclusivo` falhar e, depois de desfeita a alteração, passar novamente.
3. Você parou o programa em um ponto de parada e avaliou uma expressão com Alt+F8.

13.10 Erros frequentes e soluções

Erro	Causa	Solução
Não aparece ▶ ao lado dos testes	Executor de testes não é o pytest	Settings > Tools > Python Integrated Tools > Default test runner: pytest
"No tests were found"	Arquivo ou função sem prefixo `test_`	Renomeie
`ModuleNotFoundError` no teste	Working directory ou caminho de importação	Template de pytest com `$ProjectFileDir$` (módulo 1); confira o `sys.path.insert`
Teste passa mesmo com código errado	Teste fraco (sem `assert` útil)	Faça o "teste do teste"
Depurador não para no ponto marcado	Executou com Run, não com Debug	Use Debug (□)
Testes de Selenium marcados como "skipped"	Chrome indisponível	Instale o Chrome

13.11 Exercícios práticos

1. Escreva um teste para o seu `indicador_chuva` (módulo 9).
2. Escreva um teste para `celsius_para_fahrenheit(0) == 32`.
3. Provoque um erro (divida por zero em `media([])`), clique no link do traceback na janela Run e identifique a linha.

13.12 Critérios de conclusão

- ☐ `test_m09_regras.py` com 5 testes aprovados.
- ☐ Suíte completa executada pelo PyCharm.
- ☐ Você fez o "teste do teste" e usou o depurador.

13.13 Desafios de extensão

- Use `@pytest.mark.parametrize` para testar vários limites em um só teste (o PyCharm mostra cada variação na árvore).
- Leia `tests/conftest.py` e explique como o `FakeHttp` substitui a internet.

Módulo 14 — Documentação e versionamento com Git no PyCharm

14.1 Objetivo de aprendizagem

Versionar o projeto com **Git** usando a integração do PyCharm (repositório, *commits*, histórico, ramos, envio ao GitHub) e conhecer a documentação técnica do ClimaAlerta.

14.2 Pré-requisitos

Módulos anteriores. Git instalado (Windows: git-scm.com; Linux: `sudo apt install git`). Conta no GitHub para o envio (opcional).

14.3 Conceitos

- **Controle de versão:** registro do histórico de alterações; permite voltar atrás e trabalhar em equipe.
- **Repositório, *commit*, ramo (*branch*):** pasta versionada; "foto" das alterações com mensagem; linha de desenvolvimento paralela.
- `.gitignore`**:** lista o que **não** deve ser versionado (`.venv`, banco local, `.env`, logs). O PyCharm mostra esses arquivos em cinza/amarelo-escuro.
- **Cores dos arquivos no PyCharm:** verde = novo (ainda não versionado no último commit), azul = alterado, vermelho = não adicionado ao Git, cinza = ignorado.
- **Documentação técnica:** requisitos, regras de negócio, arquitetura, dicionário de dados, BPMN, fluxogramas, casos de uso, testes, implantação, segurança, rastreabilidade e manual técnico — cada um com uma finalidade (veja a apresentação "Documentação técnica").

14.4 Arquivos a criar ou alterar

Arquivo	Ação
`.git/`	Criado ao ativar o controle de versão
`.gitignore`	Já existe; conferir

14.5 Passo a passo no PyCharm

1. **Identificação (uma vez por computador):** no Terminal do PyCharm, informe nome e e-mail usados nos *commits*:

Terminal do PyCharm — qualquer pasta

```
git config --global user.name "Seu Nome"
git config --global user.email "seu.email@exemplo.com"
```

1. **Ativar o Git no projeto:** menu **VCS > Enable Version Control Integration...** > escolha **Git** > OK. (Em projetos clonados do GitHub, o Git já está ativo; o menu aparece como **Git**.)
2. **Primeiro commit:** abra a janela **Commit** (Ctrl+K ou Alt+0). Marque **Unversioned Files** (arquivos ainda não versionados), confira que `.venv` e `data/climaalerta.db` **não** aparecem (estão no `.gitignore`), escreva a mensagem `Primeira versão do ClimaAlerta com tutorial` e clique em **Commit**.
3. **Histórico:** abra a janela **Git** (Alt+9), aba **Log**, para ver os *commits*. Clique em um *commit* para ver os arquivos alterados.
4. **Ver diferenças:** altere uma linha de `m09_regras.py`; o PyCharm marca a linha na margem. Clique com o botão direito no arquivo > **Git > Show Diff** para comparar com a versão do último *commit*.

5. **Ramo (branch):** clique no nome do ramo na barra superior (ou **Git > New Branch...**), crie `indicador-uv`, faça um *commit* nele, volte ao ramo principal (**Checkout**) e use **Git > Merge...** para trazer as alterações.
6. **Enviar ao GitHub: Git > GitHub > Share Project on GitHub** cria o repositório na sua conta e envia o projeto. Depois, a cada *commit*, use **Git > Push** (Ctrl+Shift+K). Para receber alterações de colegas, **Git > Update Project** (Ctrl+T).

Equivalente no Terminal do PyCharm:

Terminal do PyCharm — pasta raiz

```
git init
git status
git add .
git commit -m "Primeira versão do ClimaAlerta com tutorial"
git log --oneline
git switch -c indicador-uv
```

> **Nota:** O ramo inicial pode se chamar `main` ou `master`, conforme a configuração do Git. O nome aparece na barra superior do PyCharm.

Documentação técnica: os PDFs estão em `documentacao/tecnica/` e `documentacao/pedagogica/`. No projeto completo do professor, eles são regenerados com:

Terminal do PyCharm — pasta raiz (projeto completo do professor)

```
python scripts/gerar_documentacao.py
python scripts/gerar_apresentacao.py
python scripts/validar_entrega.py
```

> **Nota:** Esses três scripts pertencem ao projeto completo do professor e não fazem parte do repositório-esqueleto dos alunos.

14.6 Código

O `.gitignore` do projeto:

.gitignore

```
# Ambiente virtual e caches
.venv/
venv/
__pycache__/
*.py[cod]
.pytest_cache/
.ruff_cache/
.coverage
htmlcov/
# Configuração local (pode conter ajustes pessoais)
.env
# Dados gerados em tempo de execução
data/*.db
data/*.db-journal
data/scraper_demo_resultado.csv
data/exportacoes/
logs/*.log*
# Sistema operacional / editores
.DS_Store
Thumbs.db
.vscode/
.idea/
# Arquivos gerados pelo tutorial
exemplos/tutorial/tutorial.db
```

.gitignore

```
exemplos/tutorial/resumo_diario.csv
```

14.7 Explicação do código

- `.venv/`, `__pycache__/` e caches são recriáveis — não vão para o repositório.
- `.env` pode conter configurações locais; `data/*.db` e `logs/` mudam a cada execução.
- Na janela Commit, apenas os arquivos marcados entram no *commit*; a mensagem deve explicar **o que e por quê**.

14.8 Resultado esperado

A janela Git > Log mostra o *commit* com a sua mensagem. Depois de **Share Project on GitHub**, o repositório aparece na sua conta do GitHub com todos os arquivos, exceto os ignorados.

14.9 Verificação

Na janela Commit não deve haver alterações pendentes logo após o *commit*. No Terminal, `git status` mostra "nothing to commit, working tree clean" e `git ls-files` **não** lista `.venv` nem `data/climaalerta.db`.

14.10 Erros frequentes e soluções

Erro	Causa	Solução
"Please tell me who you are"	Nome/e-mail não configurados	Passo 1
Menu VCS sem a opção de ativar	Git não instalado ou não encontrado	Settings > Version Control > Git: indique o `git.exe` e clique em Test
`.venv` aparece na janela Commit	`.gitignore` ausente ou com erro	Confira o arquivo na pasta raiz
Push recusado ("rejected")	O GitHub tem *commits* que você não tem	Git > Update Project (Ctrl+T) e depois Push
Conflito no Merge	Mesma linha alterada nos dois ramos	O PyCharm abre a ferramenta de mesclagem: escolha as linhas e clique em Apply

14.11 Exercícios práticos

1. Faça três *commits* pequenos com mensagens claras e veja o histórico na janela Git.
2. Abra `documentacao/tecnica/11_matriz_de_rastreabilidade.pdf` e encontre o teste que comprova o RF014.
3. Escolha um requisito do documento 01 e localize, na matriz, o módulo de código que o implementa; abra o módulo no PyCharm com Ctrl+Shift+N.

14.12 Critérios de conclusão

- ☐ Git ativado no projeto, com pelo menos um *commit*.
- ☐ Você usou Show Diff e a janela Log.
- ☐ (Opcional) Projeto enviado ao GitHub.

14.13 Desafios de extensão

- Crie um ramo para cada exercício e una-os ao principal com Merge.
- Escreva um arquivo `CHANGELOG.md` registrando as mudanças de cada *commit* importante.

Módulo 15 — Integração final e apresentação do projeto

15.1 Objetivo de aprendizagem

Concluir uma extensão do sistema de ponta a ponta (código, teste, configuração e documentação), validar a entrega e apresentar o projeto explicando arquitetura, fontes de dados, limitações e a diferença entre alertas oficiais e indicadores.

15.2 Pré-requisitos

Todos os módulos anteriores.

15.3 Conceitos

- **Definição de pronto:** uma funcionalidade só está pronta com código, testes aprovados, configuração documentada e verificação real.
- **Demonstração técnica:** mostrar o sistema funcionando, não apenas slides.
- **Honestidade técnica:** apresentar limitações (fontes externas, previsões, testes só em Windows etc.).
- **Refatoração segura:** os testes existentes garantem que a mudança não quebrou o que já funcionava.

15.4 Arquivos a criar ou alterar

Projeto final: **indicador IND08 — Índice UV elevado**, uma regra diária que dispara quando o índice UV máximo previsto for maior ou igual ao limite (padrão 8).

Arquivo	O que muda
`src/config.py`	Classe `IndicatorThresholds` (novo campo) e função `get_settings` (lê o novo limite)
`src/services/indicators.py`	Função `build_rules` (nova regra IND08)
`.env.example` e `.env`	Nova variável `LIMITE_UV_MAXIMO`
`tests/test_indicators.py`	Um teste atualizado e um teste novo

15.5 Passo a passo no PyCharm

1. Crie um ramo `indicador-uv` (módulo 14, passo 6).
2. Abra `src/config.py` com **Ctrl+Shift+N**. Use **Ctrl+F12** para localizar a classe `IndicatorThresholds` e substitua-a **inteira** pela versão da seção 15.6 (código 1). Depois localize `get_settings` e substitua-a **inteira** pelo código 2.
3. Abra `src/services/indicators.py` e substitua a função `build_rules` **inteira** pelo código 3.
4. Abra `.env.example` e acrescente, ao final do bloco de limites, as linhas do código 4. Faça o mesmo no seu `.env`.
5. Abra `tests/test_indicators.py`: substitua a função `test_rules_are_built_from_thresholds` pelo código 5 e acrescente, no final do arquivo, a função do código 6.
6. Pressione **Ctrl+Alt+L** em cada arquivo alterado e confira que não há sublinhados vermelhos.
7. Clique com o botão direito em `tests/test_indicators.py` > Run. Depois, na pasta `tests` > **Run 'pytest in tests'** (suíte completa).
8. Execute a configuração `ClimaAlerta`, clique em "Atualizar agora" e abra a Central de alertas: os dias com UV ≥ 8 aparecem na seção de **indicadores calculados**.
9. Faça o *commit* (Ctrl+K) e o Merge no ramo principal.

Equivalente no Terminal do PyCharm (verificação final):

Terminal do PyCharm — pasta raiz

```
ruff check .
pytest -v
streamlit run app.py
```

15.6 Código

Os trechos abaixo são as **versões completas** das classes e funções alteradas. As linhas novas estão marcadas com o comentário `# NOVO (IND08)`. Este código foi verificado automaticamente: ele é aplicado a uma cópia do projeto e a suíte de testes é executada (`tests/test_projeto_final.py`).

Código 1 — `src/config.py`, classe `IndicatorThresholds` completa:

src/config.py — classe IndicatorThresholds (substitui a classe inteira)

```
@dataclass(frozen=True)
class IndicatorThresholds:
    """Limites dos indicadores locais.
    IMPORTANTE: são parâmetros DEMONSTRATIVOS para fins didáticos, e não
    recomendações oficiais de segurança.
    """
    precipitation_probability: float = 70.0  # %
    precipitation_hourly_mm: float = 10.0  # mm em 1 hora
    precipitation_daily_mm: float = 30.0  # mm no dia
    wind_gusts_kmh: float = 60.0  # km/h
    temperature_max_c: float = 35.0  # °C
    temperature_min_c: float = 3.0  # °C
    humidity_min_percent: float = 30.0  # %
    # NOVO (IND08): índice UV máximo do dia. Na escala da OMS, 8 a 10 é "muito alto".
    # Como todos os campos desta classe, o valor é DEMONSTRATIVO e pode ser
    # alterado no .env pela variável LIMITE_UV_MAXIMO.
    uv_index_max: float = 8.0  # índice UV
```

Código 2 — `src/config.py`, função `get_settings` completa:

src/config.py — função get_settings (substitui a função inteira)

```
@lru_cache(maxsize=1)
def get_settings() -> Settings:
    """Lê o arquivo .env (se existir) e devolve as configurações."""
    load_dotenv(PROJECT_ROOT / ".env", override=False)
    thresholds = IndicatorThresholds(
        precipitation_probability=_env_float("LIMITE_PROBABILIDADE_CHUVA", 70.0),
        precipitation_hourly_mm=_env_float("LIMITE_PRECIPITACAO_HORARIA_MM", 10.0),
        precipitation_daily_mm=_env_float("LIMITE_PRECIPITACAO_DIARIA_MM", 30.0),
        wind_gusts_kmh=_env_float("LIMITE_RAJADA_KMH", 60.0),
        temperature_max_c=_env_float("LIMITE_TEMPERATURA_MAXIMA_C", 35.0),
        temperature_min_c=_env_float("LIMITE_TEMPERATURA_MINIMA_C", 3.0),
        humidity_min_percent=_env_float("LIMITE_UMIDADE_MINIMA", 30.0),
        # NOVO (IND08): lê LIMITE_UV_MAXIMO do .env; se não existir ou for inválido, usa 8.0.
        uv_index_max=_env_float("LIMITE_UV_MAXIMO", 8.0),
    )
    return Settings(
        database_path=_resolve_path(_env_str("CLIMAALERTA_DB_PATH", "data/climaalerta.db")),
        log_dir=_resolve_path(_env_str("CLIMAALERTA_LOG_DIR", "logs")),
        log_level=_env_str("CLIMAALERTA_LOG_LEVEL", "INFO").upper(),
        timezone=_env_str("CLIMAALERTA_TIMEZONE", "America/Sao_Paulo"),
        http_timeout_seconds=_env_float("HTTP_TIMEOUT_SEGUNDOS", 15.0),
        http_max_retries=max(0, min(_env_int("HTTP_MAX_RETENTATIVAS", 3), 5)),
        http_backoff_factor=_env_float("HTTP_BACKOFF", 1.0),
        user_agent=_env_str("HTTP_USER_AGENT", "ClimaAlerta/1.0 (projeto educacional; uso local)"),
        update_interval_minutes=max(15, _env_int("INTERVALO_ATUALIZACAO_MINUTOS", 60)),
        stale_after_minutes=max(30, _env_int("DADOS_DESATUALIZADOS_APOS_MINUTOS", 180)),
        forecast_days=max(1, min(_env_int("DIAS_PREVISAO", 7), 16)),
        open_meteo_forecast_url=_env_str("OPEN_METEO_FORECAST_URL", "https://api.open-meteo.com/v1/forecast"),
        open_meteo_geocoding_url=_env_str("OPEN_METEO_GEOCODING_URL", "https://geocoding-api.open-meteo.com/v1/search"),
        inmet_alerts_url=_env_str("INMET_AVISOS_URL", "https://apiprevmet3.inmet.gov.br/avisos/ativos"),
```

src/config.py — função get_settings (substitui a função inteira)

```
        inmet_alert_page_url=_env_str("INMET_AVISO_PAGINA_URL", "https://avisos.inmet.gov.br/{id}"),
        ibge_municipalities_url=_env_str(
            "IBGE_MUNICIPIOS_URL",
            "https://servicodados.ibge.gov.br/api/v1/localidades/estados/{uf}/municipios",
        ),
        selenium_headless=_env_bool("SELENIUM_HEADLESS", True),
        selenium_timeout_seconds=_env_float("SELENIUM_TIMEOUT_SEGUNDOS", 10.0),
        thresholds=thresholds,
    )
```

Código 3 — `src/services/indicators.py`, função `build_rules` completa:

src/services/indicators.py — função build_rules (substitui a função inteira)

```
def build_rules(t: IndicatorThresholds) -> list[IndicatorRule]:
    """Cria as regras a partir dos limites configurados."""
    return [
        IndicatorRule(
            "IND01",
            "Probabilidade elevada de precipitação",
            "precipitation_probability",
            ">=",
            t.precipitation_probability,
            "%",
            "HORARIA",
        ),
        IndicatorRule(
            "IND02",
            "Precipitação horária acima do limite",
            "precipitation",
            ">=",
            t.precipitation_hourly_mm,
            "mm",
            "HORARIA",
        ),
        IndicatorRule(
            "IND03",
            "Precipitação diária acima do limite",
            "precipitation_sum",
            ">=",
            t.precipitation_daily_mm,
            "mm",
            "DIARIA",
        ),
        IndicatorRule(
            "IND04", "Rajadas de vento acima do limite", "wind_gusts", ">=", t.wind_gusts_kmh, "km/h", "HORARIA"
        ),
        IndicatorRule(
            "IND05", "Temperatura máxima elevada", "temperature_max", ">=", t.temperature_max_c, "°C", "DIARIA"
        ),
        IndicatorRule(
            "IND06", "Temperatura mínima baixa", "temperature_min", "<=", t.temperature_min_c, "°C", "DIARIA"
        ),
        IndicatorRule("IND07", "Umidade relativa baixa", "humidity", "<=", t.humidity_min_percent, "%", "HORARIA"),
        # NOVO (IND08): regra DIÁRIA, porque a previsão diária da Open-Meteo já traz o
        # "uv_index_max" (coluna uv_index_max da tabela weather_forecast).
        # Operador ">=" (limite inclusivo) e unidade vazia, pois o índice UV não tem unidade.
        IndicatorRule("IND08", "Índice UV elevado", "uv_index_max", ">=", t.uv_index_max, "", "DIARIA"),
    ]
```

Código 4 — `.env.example` (e `.env`):

.env.example — linhas a acrescentar

```
# NOVO (IND08) — acrescente ao final do bloco de limites dos indicadores:
LIMITE_UV_MAXIMO=8
```

Código 5 — `tests/test_indicators.py`, teste atualizado:

tests/test_indicators.py — substitui test_rules_are_built_from_thresholds

```
def test_rules_are_built_from_thresholds():
    rules = build_rules(IndicatorThresholds(wind_gusts_kmh=50))
    assert {r.rule_id for r in rules} == {f"IND0{i}" for i in range(1, 9)}  # agora são 8 regras: IND01 a IND08
```

tests/test_indicators.py — substitui test_rules_are_built_from_thresholds

```
    assert next(r for r in rules if r.rule_id == "IND04").threshold == 50
```

Código 6 — `tests/test_indicators.py`, teste novo:

tests/test_indicators.py — acrescentar no final do arquivo

```
def test_uv_index_rule():
    """IND08: dispara no dia com UV >= 8 e ignora o dia com UV 6."""
    # Procura a regra IND08 entre as regras criadas com os limites padrão.
    rule = next(r for r in build_rules(IndicatorThresholds()) if r.rule_id == "IND08")
    # day() é a função auxiliar do próprio arquivo de testes: cria uma linha de previsão diária.
    hits = evaluate_daily(rule, [day(21, uv_index_max=9.1), day(22, uv_index_max=6.0)])
    # Só o dia 21 atingiu o limite.
    assert [h.reference_time for h in hits] == [date(2026, 10, 21)]
    assert hits[0].value == 9.1 and hits[0].classification.startswith("INDICADOR CALCULADO")
```

15.7 Explicação do código

- **Código 1:** o novo campo `uv_index_max` tem valor padrão 8.0. Como a classe é imutável (`frozen=True`), o valor só muda ao criar uma nova instância — feito por `get_settings`.
- **Código 2:** `_env_float("LIMITE_UV_MAXIMO", 8.0)` lê a variável do `.env`; se ela não existir ou não for um número, usa 8.0.
- **Código 3:** a regra é **diária** porque a previsão diária já traz `uv_index_max`; o operador `>=` torna o limite inclusivo; a unidade é vazia porque o índice UV não tem unidade.
- **Código 4:** o teste `test_env_example_documents_every_variable` confere que toda variável lida no `config.py` está documentada no `.env.example`. Esquecer este passo faz a suíte falhar — a documentação faz parte da definição de pronto.
- **Código 5:** o teste antigo esperava exatamente 7 regras (IND01 a IND07); agora são 8.
- **Código 6:** o teste novo usa as funções auxiliares `day()` e `evaluate_daily`, que já existem no arquivo, e confere que só o dia com UV 9,1 dispara o indicador.

15.8 Resultado esperado

- `ruff check .`: "All checks passed!".
- Suíte completa: todos os testes aprovados, incluindo `test_uv_index_rule`.
- Na Central de alertas, os dias com UV ≥ 8 aparecem como **indicadores calculados** (nunca como alertas oficiais). Na previsão real de 21/10/2026 para Sapiranga, os índices UV máximos variavam de 4,25 a 8,3; com o limite 8, apenas os dias com UV ≥ 8 apareceriam.

15.9 Verificação

Use o checklist da seção 15.12 e o roteiro de apresentação abaixo.

Roteiro de apresentação (10 a 15 minutos):

1. Problema e objetivo do ClimaAlerta (1 min).
2. Demonstração ao vivo pelo PyCharm (configuração `ClimaAlerta`): Painel, Previsão, Central de alertas — destacando oficial × calculado (4 min).
3. Arquitetura em camadas e fontes de dados (2 min) — use o diagrama do documento 03.
4. Selenium na página de demonstração e por que o INMET é consumido via API (2 min).
5. Testes na janela do PyCharm (árvore verde) e matriz de rastreabilidade (2 min).
6. A extensão feita pelo grupo (IND08 ou outra) (2 min).
7. Limitações e próximos passos (1 min).

15.10 Erros frequentes e soluções

Erro	Causa	Solução
`test_env_example_documents_every_variable` falhou	Variável nova não documentada	Código 4 no `.env.example`
`test_rules_are_built_from_thresholds` falhou	Teste ainda espera 7 regras	Código 5
`TypeError: IndicatorThresholds.__init__() got an unexpected keyword argument 'uv_index_max'`	`get_settings` alterada, mas a classe não	Código 1
Indicador novo não aparece na interface	Interface não reiniciada ou dados antigos	Pare (■), execute de novo e clique em "Atualizar agora"
Internet falha durante a apresentação	Rede instável	O sistema mostra os últimos dados do banco; explique a regra RN010

15.11 Exercícios práticos

1. Implemente o IND08 seguindo os passos.
2. Altere `LIMITE_UV_MAXIMO` para 7 no `.env`, reinicie e compare a quantidade de indicadores.
3. Prepare a apresentação usando o roteiro e avalie o trabalho de outro grupo com o checklist.

15.12 Critérios de conclusão (checklist final)

- ☐ Interpretador `.venv` configurado e dependências instaladas pelo PyCharm.
- ☐ Todos os arquivos do tutorial (m01 a m12 e test_m09) criados e testados.
- ☐ ClimaAlerta executado com coleta real bem-sucedida.
- ☐ IND08 implementado com testes aprovados.
- ☐ `ruff check .` sem erros e suíte completa 100 % aprovada.
- ☐ Repositório Git com histórico de *commits* (e, se possível, no GitHub).
- ☐ Apresentação explica a diferença entre alerta oficial e indicador calculado.

15.13 Desafios de extensão

- Notificar na barra lateral quando surgir um novo aviso oficial para o município principal.
- Consultar a previsão de vários municípios em uma única requisição (a Open-Meteo aceita listas de coordenadas).
- Configurar o GitHub Actions para executar os testes a cada *push* (o repositório-esqueleto já traz um modelo em `.github/workflows/testes.yml`).

Revisão geral

Módulo	Conceito-chave	Arquivo principal	Recurso do PyCharm
1	Ambiente virtual e dependências	`m01_verificar_ambiente.py`	Python Interpreter, Install requirements
2	Variáveis, funções, condições e laços	`m02_fundamentos.py`	Autocompletar, Ctrl+Q, Ctrl+Alt+L
3	Dashboard e widgets	`m03_primeira_dashboard.py`	Run Configuration de módulo
4	HTML, DOM e seletores CSS	`m04_pagina.html`	Abrir no navegador
5	Selenium e espera explícita	`m05_selenium_basico.py`	Pontos de parada, F8
6	API REST, JSON e timeout	`m06_api_open_meteo.py`	Inspeção de variáveis no Debug
7	DataFrame, filtro e agrupamento	`m07_pandas.py`	View as DataFrame
8	ORM, chaves e restrições	`m08_banco.py`	Visualização do banco
9	Regras de negócio e indicadores	`m09_regras.py`	Python Console
10	Interface que lê o banco	`m10_interface.py`	Sources Root
11	Arquitetura em camadas	`app.py`, `src/`	Ctrl+B, Alt+F7, janela TODO
12	Automação robusta	`m12_agendador.py`	Script parameters
13	Testes e depuração	`test_m09_regras.py`	Árvore de testes, depurador
14	Git e documentação	`.gitignore`, `documentacao/`	Commit, Log, Share on GitHub
15	Definição de pronto e apresentação	Extensão IND08	Ramos, Merge, suíte completa

Glossário

Termo	Significado
API	Interface para programas trocarem dados (aqui, via HTTP e JSON)
Backoff	Espera crescente entre tentativas após uma falha
Breakpoint	Ponto de parada do depurador
CSV	Arquivo de texto com colunas separadas por um caractere (`;` no projeto)
DataFrame	Tabela do Pandas
Headless	Navegador executado sem janela
IDE	Ambiente de desenvolvimento integrado (aqui, o PyCharm)
Idempotente	Executar várias vezes produz o mesmo resultado que executar uma
Indicador local	Cálculo do sistema com limite configurável — não oficial
Interpretador do projeto	O Python (do `.venv`) que o PyCharm usa para executar o projeto
ORM	Mapeamento entre classes Python e tabelas do banco
Run Configuration	Configuração salva no PyCharm para executar um programa
Sources Root	Pasta que o PyCharm considera raiz de importações
Timeout	Tempo máximo de espera de uma operação
Upsert	Inserir ou, se já existir, atualizar
UTC	Horário universal; Brasília = UTC−3
WMO	Organização Meteorológica Mundial; seus códigos descrevem a condição do tempo

Referências

- PyCharm — documentação oficial: https://www.jetbrains.com/help/pycharm/
- PyCharm — configurar interpretador virtualenv: https://www.jetbrains.com/help/pycharm/creating-virtual-environment.html
- PyCharm — testes com pytest: https://www.jetbrains.com/help/pycharm/pytest.html
- PyCharm — depuração: https://www.jetbrains.com/help/pycharm/debugging-code.html
- PyCharm — Git: https://www.jetbrains.com/help/pycharm/using-git-integration.html
- Python — documentação oficial: https://docs.python.org/pt-br/3/
- Streamlit — documentação: https://docs.streamlit.io/
- Plotly Express: https://plotly.com/python/plotly-express/
- Selenium — documentação: https://www.selenium.dev/documentation/
- Requests: https://requests.readthedocs.io/
- Pandas — guia do usuário: https://pandas.pydata.org/docs/user_guide/
- SQLAlchemy 2.0 — tutorial: https://docs.sqlalchemy.org/en/20/tutorial/
- pytest: https://docs.pytest.org/
- Ruff: https://docs.astral.sh/ruff/
- Git — livro Pro Git (português): https://git-scm.com/book/pt-br/v2
- Open-Meteo — API de previsão: https://open-meteo.com/en/docs
- Open-Meteo — geocodificação: https://open-meteo.com/en/docs/geocoding-api
- INMET — avisos meteorológicos: https://avisos.inmet.gov.br/
- IBGE — API de localidades: https://servicodados.ibge.gov.br/api/docs/localidades
- MDN — seletores CSS: https://developer.mozilla.org/pt-BR/docs/Web/CSS/CSS_selectors
- Documentação técnica do ClimaAlerta: `documentacao/tecnica/` (documentos 01 a 12)